



УНИВЕРЗИТЕТ У НИШУ
ЕЛЕКТРОНСКИ ФАКУЛТЕТ



**Математичко моделовање система за псеудо-случајно
генерисање токена и евалуацију комбинација над
дискретним матрицама**

Завршни рад

Студијски програм: Електротехника и рачунарство

Модул: Рачунарство и информатика

Студент:
Јован Богдановић
Бр. инд. 19059

Ментор:
Доц. др Петар Рајковић

Ниш, јул 2026.

Универзитет у Нишу
Електронски Факултет

Математичко моделовање система за псеудо-случајно генерисање токена и евалуацију комбинација над дискретним матрицама

Mathematical modeling of a system for pseudo-random token generation and the evaluation of combinations over discrete matrices

Завршни рад
Студијски програм: Електротехника и рачунарство
Модул: Рачунарство и информатика

Студент: Јован Богдановић бр. инд. 19059

Ментор: Доц. др Петар Рајковић

Задатак: Математичко моделовање система за псеудо-случајно генерисање и евалуацију комбинација токена над дискретним матрицама, укључујући конструкцију генеративне матрице за задати улазно-излазни однос, реконструкцију непознате матрице из посматраних итерација применом Де Брујнових графова и компаративну анализу алгоритама за евалуацију комбинација.

Датум пријаве рада: 10.7.2026.

Датум предаје рада: 17.7.2026.

Датум одбране рада: 23.7.2026.

Комисија за оцену и одбрану:

Доц. др Петар Рајковић, Председник Комисије

Доц. др Милена Фртунић Глигоријевић, Члан

Доц. др Александар Миленковић, Члан

Математичко моделовање система за псеудо-случајно генерисање токена и евалуацију комбинација над дискретним матрицама

Сажетак

Предмет овог рада је математичко моделовање и анализа система за псеудо-случајно генерисање токена и евалуацију комбинација над дискретним матрицама. Систем се састоји од генеративне компоненте, која на основу цикличних токен-секвенци псеудо-случајно производи матрицу токена, и евалуационе компоненте, која применом дефинисаних правила тој матрици додељује нумеричку излазну вредност.

У раду је најпре уведен формални модел система са комплетном терминологијом и нотацијом, укључујући дефиниције генеративне матрице, видљивог прозора, евалуационих путања, стандардних и специјалних токена, и улазно-излазног односа система.

Затим је обрађен проблем конструкције генеративне матрице за задати улазно-излазни однос, где се комбинацијом нелинеарне оптимизације методом SLSQP, дискретизације методом највећег остатка, исцрпне енумерације и Поасон-биномијалног динамичког програмирања долази до конкретне дискретне структуре са контролисаним статистичким својствима.

Проблем реконструкције непознате генеративне матрице из посматраних итерација моделиран је помоћу Де Брајнових графова, где се посматрани фрагменти видљивог прозора користе за изградњу графа чијим обиласком — проналажењем Ојлеровог циклуса — се добија оригинална циклична токен-секвенца, уз разрешавање проблема дупликата статистичким скалирањем.

На крају је извршена компаративна анализа три алгорита за евалуацију комбинација — секвенцијалне евалуације, евалуације помоћу хеш табеле и евалуације помоћу индексног стабла — са становишта временске и меморијске сложености, при чему је експериментална верификација на до 100 милиона итерација показала да индексно стабло остварује конзистентно убрзање од 1.5x до 2x захваљујући дељењу заједничких префикса и раном одсецању грана.

Mathematical modeling of a system for pseudo-random token generation and the evaluation of combinations over discrete matrices

Abstract

This thesis presents the mathematical modeling and analysis of a system for pseudo-random token generation and combination evaluation over discrete matrices. The system consists of a generative component, which pseudo-randomly produces a token matrix from cyclic token sequences, and an evaluation component, which assigns a numerical output value to that matrix according to defined rules.

The thesis first introduces a formal system model with complete terminology and notation, including definitions of the generative matrix, visible window, evaluation paths, standard and special tokens, and the system input-output ratio.

The problem of constructing a generative matrix for a given input-output ratio is then addressed, where a combination of nonlinear optimization using the SLSQP method, discretization using the largest remainder method, exhaustive enumeration, and Poisson-binomial dynamic programming yields a concrete discrete structure with controlled statistical properties.

The problem of reconstructing an unknown generative matrix from observed iterations is modeled using De Bruijn graphs, where observed fragments of the visible window are used to build a graph whose traversal — by finding an Eulerian cycle — recovers the original cyclic token sequence, with the duplicate problem resolved through statistical scaling.

Finally, a comparative analysis of three combination evaluation algorithms is performed — sequential evaluation, hash table evaluation, and index tree (trie) evaluation — in terms of time and space complexity, where experimental verification on up to 100 million iterations demonstrated that the index tree achieves a consistent speedup of 1.5x to 2x due to shared prefix traversal and early branch pruning.

Садржај

1. Увод.....	7
2. Формални модел система.....	8
2.1. Уводне напомене	8
2.2. Азбука токена	8
2.3. Циклична токен-секвенца.....	8
2.4. Генеративна матрица	9
2.5. Псеудо-случајни генератор (ПСГ).....	9
2.6. Итерација система	10
2.7. Видљиви прозор	11
2.8. Евалуациона путања.....	11
2.9. Универзални заменски токен	12
2.10. Позиционо-инваријантан токен	12
2.11. Активациони токен	13
2.12. Евалуациона функција и табела излазних вредности итерације	14
2.13. Укупна излазна вредност једне итерације система.....	15
2.14. Улазна вредност итерације	15
2.15. Улазно-излазни однос система.....	15
2.16. Варијанса система	16
3. Конструкција генеративне матрице са контролисаним улазно-излазним односом	17
3.1. Увод и формулација инверзног проблема	17
3.2. Густина токена, улаз и излаз конструкције	18
3.3. Очекивана излазна вредност стандардних токена	18
3.4. Оптимизација густине токена	20
3.5. Дискретизација	23
3.6. Укључивање специјалних токена	24
3.7. Укупни улазно-излазни однос и финалне провере	28
3.8. Генерисање цикличне токен-секвенце	28
3.9. Закључак поглавља.....	29
4. Реконструкција цикличне токен-секвенце помоћу Де Брујнових графова.....	30
4.1. Уводне напомене и формулација проблема.....	30
4.2. Аналогија са биоинформатиком	30
4.3. Дефиниција модела	31

4.4. Изградња графа из посматраних узорака.....	32
4.5. Ојлеров циклус и реконструкција секвенце	33
4.6. Проблем дупликата и његово решење.....	35
4.7. Ограничења и напомене.....	37
4.8. Закључак поглавља.....	38
5. Компаративна анализа алгоритма за евалуацију комбинација	39
5.1. Уводне напомене	39
5.2. Секвенцијална евалуација	39
5.3. Евалуација помоћу хеш табеле	44
5.4. Евалуација помоћу индексног стабла.....	48
5.5. Компаративна анализа	56
5.6. Закључак поглавља.....	58
6. Литература.....	59

1. Увод

Псеудо-случајно генерисање дискретних секвенци над коначним скупом елемената представља фундаменталан проблем у више области рачунарских наука и примењене математике. Системи који комбинују стохастичко генерисање са детерминистичком евалуацијом комбинација појављују се у контекстима који обухватају тестирање стохастичких алгоритама, анализу дискретних дистрибуција и пројектовање система за процену ризика. Иако се конкретне примене разликују, математичка структура ових система је заједничка: постоји унапред дефинисана структура података из које се псеудо-случајно бирају елементи, и постоји скуп правила по којима се генерисани резултат евалуира.

Предмет овог рада је класа система које називамо дискретним генераторима са евалуацијом комбинација. Овакав систем се састоји од две компоненте: генеративне, која на основу цикличних токен-секвенци псеудо-случајно инстанцира матрицу токена, и евалуационе, која применом дефинисаних правила тој матрици додељује нумеричку излазну вредност. Централна структура система је генеративна матрица — уређена колекција цикличних токен-секвенци над коначним скупом токена. Распоред токена унутар ових секвенци у потпуности одређује статистичко понашање система: очекивану излазну вредност, варијансу и комплетну дистрибуцију резултата.

Овакав модел отвара три природна истраживачка питања. Прво, како конструисати генеративну матрицу тако да улазно-излазни однос система — однос очекиване излазне и улазне вредности — тачно одговара задатој вредности? Ово је инверзни проблем у којем се од жељеног статистичког својства долази до конкретне дискретне структуре, а његово решавање захтева комбинацију континуалне оптимизације, дискретизације и прецизне евалуације доприноса специјалних токена. Друго, ако генеративна матрица није позната, да ли је могуће реконструисати је из великог броја посматраних итерација? Овај проблем се природно моделира помоћу Де Брујнових графова, где се кратки посматрани фрагменти уланчавају у комплетну секвенцу проналажењем Ојлеровог циклуса. Треће, који алгоритам за евалуацију комбинација у генерисаној матрици даје оптималне перформансе? Одговор на ово питање захтева компаративну анализу различитих приступа — од директне секвенцијалне евалуације, преко претраге хеш табеле, до обиласка индексног стабла — са становишта временске и меморијске сложености.

Рад се ослања на резултате из неколико математичких и рачунарских области. Теорија вероватноће обезбеђује основу за израчунавање очекиваних вредности и анализу конвергенције. Нелинеарна оптимизација омогућава проналажење оптималних густина токена за задати улазно-излазни однос. Теорија графова пружа елегантан модел за проблем реконструкције. Анализа алгоритама и структуре података дају оквир за ригорозно поређење евалуационих приступа. Повезивање ових области у јединствен формални модел представља централни допринос овог рада.

2. Формални модел система

2.1. Уводне напомене

Систем који се анализира у овом раду састоји се од две јасно раздвојене компоненте: генеративне и евалуационе. Генеративна компонента на основу унапред припремљене структуре података псеудо-случајно производи матрицу токена. Евалуациона компонента ту матрицу анализира према дефинисаним правилима и израчунава нумерички резултат.

Ово поглавље уводи све појмове и структуру података која чини основу система. Сваки појам је објашњен са становишта тога шта представља, како је имплементиран и какву улогу има у раду система. Формална нотација се уводи тамо где је потребна за прецизност, али фокус је на разумевању система као целине.

2.2. Азбука токена

Азбука токена је коначни скуп различитих елемената из којих се граде све секвенце у систему:

$$\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_k\}$$

Сваки елемент овог система називамо **токен**. Токен је основна, недељива јединица система – аналогно карактеру у стрингу или елементу енумерације у програмском језику. Број токена у азбуци означавамо са $k = |\Sigma|$.

У практичним системима, k се најчешће креће у опсегу од 8 до 15. Мања азбука значи већу вероватноћу поклапања токена (јер има мање дискретних вредности), док већа азбука даје разноврснији простор комбинација.

Пример. Азбука $\Sigma = \{A, B, C, D, E, F, G, H\}$ садржи $k = 8$ токена. Ознаке су произвољне – битна је само чињеница да су међусобно различите.

2.3. Циклична токен-секвенца

Циклична токен-секвенца је низ токена фиксне дужине који се понаша као кружна структура — након последњег елемента следи први:

$$r_i = (r_0, r_1, \dots, r_{L-1})$$

где је L дужина секвенце, а сваки елемент припада алфабету: $r_i \in \Sigma$.

Из перспективе структура података, ово је **циркуларни низ** (circular buffer). Индексирање је циклично: приступ елементу на позицији i се увек рачуна као i мод L па позиција након $L - 1$ није ван границе него се враћа на 0.

Ова цикличност је кључна за рад система. Када се из секвенце чита прозор од m узастопних елемената почевши од неке позиције s , читање природно прелази границу низа:

Секвенца: [A, B, C, D, E, F] ($L = 6$)

Позиција: 0 1 2 3 4 5

Читање 3 елемената од позиције $s = 4$:

позиција 4 $\rightarrow$ E

позиција 5 $\rightarrow$ F

позиција 0 $\rightarrow$ A (wrap-around)

Резултат: [E, F, A]

Густина токена. За сваку секвенцу можемо пребројати колико пута се сваки токен појављује. Ако се токен σ појављује $c(\sigma)$ пута у секвенци r дужине L , тада је његова релативна густина:

$$p(\sigma) = \frac{c(\sigma)}{L}$$

2.4. Генеративна матрица

Генеративна матрица је централна структура података система. Она се састоји од n цикличних токен-секвенци, где свака секвенца представља једну колону:

$$M = (r_1, r_1, \dots, r_n)$$

Из програмерске перспективе, генеративна матрица је **низ низова** где сваки унутрашњи низ може имати различиту дужину. Она није матрица у класичном смислу (где све колоне морају имати исти број елемената) – то је колекција секвенци које заједно дефинишу систем.

Генеративна матрица у потпуности одређује понашање система. Свака статистичка карактеристика произлази искључиво из распореда токена унутар ове структуре. Два система са идентичном евалуационом функцијом али различитим генеративним матрицама могу имати драстично различите перформансе.

2.5. Псеудо-случајни генератор (ПСГ)

У свакој итерацији система, потребно је за сваку колону одабрати почетни индекс за који важи $s \in \{0, 1, \dots, L-1\}$ – позиција у цикличној токен-секвенци од које почиње читање видљивог прозора. Тај избор врши псеудо-случајни генератор (ПСГ).

ПСГ је детерминистички алгоритам који на основу почетне вредности (*seed*) производи секвенцу бројева са својствима статистичке непредвидивости. За разлику од правих извора случајности, ПСГ је у потпуности репродуцибилан – исти *seed* увек даје исту секвенцу. Упркос томе генерисана секвенца пролази статистичке тестове униформности

и независности, па је за потребе овог система функционално еквивалентна правој случајности.

Конкретна имплементација ПСГ-а (избор алгоритма, дужина периоде, аутокорељациона својства) није предмет овог рада. За сву анализу која следи, битне су само две претпоставке о понашању генератора:

Претпоставка униформности – свака позиција у колони има једнаку вероватноћу да буде изабрана као почетни индекс, то значи да ни једна позиција у секвенци није фаворизована – генератор не преферира одређене делове секвенце.

Претпоставка независности – избор почетног индекса у једној колони не утиче на избор у било којој другој колони.

Ове две претпоставке заједно гарантују да је свака комбинација индекса једнако вероватна. То је основа за израчунавање очекиване излазне вредности система као простог просека по свим стањима, као и за статистичку анализу реконструкције у Поглављу X.

2.6. Итерација система

Итерација је једна комплетна инстанца рада система – од генерисања до коначног нумеричког резултата. Састоји се од два корака:

Фаза генерисања

За сваку колону j од 1 до n :

1. ПСГ генерише почетни индекс $s_j \in \{0, 1, \dots, L_j - 1\}$.
2. Из цикличне токен-секвенце r_j чита се m узастопних токена од позиције s_j .

Резултат је **видљиви прозор**.

Фаза евалуације

На видљиви прозор примењују се евалуациона правила (описана у 2.8. и 2.12.) израчунава се излазна вредност итерације.

Важно је нагласити да је итерација атомска операција. Након покретања, њен исход је у потпуности детерминисан генерисаним вектором индекса $s = (s_1, s_2, \dots, s_n)$. Не постоји никаква интеракција корисника током извршавања једне итерације.

Пример – за систем са $n = 3$ колоне и прозором висине $m = 3$:

Kolona 1 ($L = 6$): [A, B, C, D, A, B] $s = 4$

Kolona 2 ($L = 5$): [C, A, B, A, D] $s = 2$

Kolona 3 ($L = 7$): [B, A, A, C, B, D, A] $s = 5$

Читање прозора:

Колона 1, од позиције 4: позиције 4,5,0 $\rightarrow$ [A, B, A]

Колона 2, од позиције 2: позиције 2,3,4 $\rightarrow$ [B, A, D]

Колона 3, од позиције 5: позиције 5,6,0 $\rightarrow$ [D, A, B]

Видљиви прозор:

	Кол. 1	Кол. 2	Кол. 3
Ред 0:	A	B	D
Ред 1:	B	A	A
Ред 2:	A	D	B

2.7. Видљиви прозор

Видљиви прозор је матрица $\mathbf{W}$ димензија $m \times n$ која настаје у фази генерисања. Елемент w_{ij} означава токен у реду i и колони j .

Видљиви прозор је једина информација доступна евалуационој компоненти. Евалуациони алгоритам не зна како изгледа генеративна матрица, нити које су вредности индекса s – ради искључиво са садржајем прозора.

Ово раздвајање је концептуално важно: генеративна матрица је трајна структура која дефинише систем, док је видљиви прозор пролазна инстанца генерисања у једној итерацији.

2.8. Евалуациона путања

Евалуациона путања дефинише које позиције у видљивом прозору се посматрају заједно приликом провере дали постоји валидан образац. Формално, то је низ од n индекса:

$$p_l = (p_{l,1}, p_{l,2}, \dots, p_{l,n}), p_{l,j} \in \{0, 1, \dots, m-1\}$$

Сваки елемент $p_{l,j}$ каже у којој колони j посматрајмо који ред. Путања дакле бира тачно једну позицију из сваке колоне.

Визуелно – за прозор димензија 3×5 , путања (0,1,2,1,0) бира следеће позиције (означено са X):

	Кол. 1	Кол. 2	Кол. 3	Кол. 4	Кол.5
Ред 0:	[X]	[]	[]	[]	[X]
Ред 1:	[]	[X]	[]	[X]	[]
Ред 2:	[]	[]	[X]	[]	[]

Токени на означеним позицијама формирају евалуациону секвенцу:

$$e_l = (w_{1,pl,1}, w_{2,pl,2}, \dots, w_{n,pl,n})$$

Ова секвенца се затим прослеђује евалуационој функцији.

2.9. Универзални заменски токен

Универзални заменски токен σ_w је специјални елемент азбуке који при евалуацији може „глумити“ било који други токен.

Конкретно, када се рачуна дужина префикса за токен σ , заменски токен се третира као да јесте σ :

$$\ell(\sigma, e) = \max\{l : \forall i \leq l, e_i \in \{\sigma, \sigma_w\}\}$$

Пример. Секвенца (A, σ_w, A, B, C) при евалуацији за токен A има префикс дужине 3 – позиције 1, 2 и 3 су све или A или σ_w , па се боје као узастопна понављања.

Заменски токен дакле проширује скуп секвенци које се препознају као валидан образац – свака позиција на којој се налази σ_w прихвата се као поклапање са било којим токеном који се тренутно евалуира (који се налази у префиксу).

2.10. Позиционо-инваријантан токен

Позиционо-инваријанта токен σ_s се евалуира на другачији начин од свих осталих. Уместо да се посматра дуж евалуационих путања, евалуација зависи од укупног броја појављивања у целом видљивом прозору, без обзира на позицију.

Дефинишемо бројач:

$$N_s(W) = \text{број позиција } (j, i) \text{ у прозору где је } w_{j,i} = \sigma_s$$

Евалуациона функција за овај токен је:

$$\psi_s(N_s) \rightarrow \mathbb{R}_{\geq 0}$$

са прагом активације N_{min} – ако је $N_s < N_{min}$, излазна вредност је 0.

Кључна разлика: Стандардна евалуација ради дуж путање и гледа n позиција (једну из сваке колоне). Евалуација позиционо-инваријантног токена скенира цео прозор и гледа

свих $m \times n$ позиција. Позиције на којима се токен налази су потпуно ирелевантне – једино је битан укупан збир.

Пример: Прозор димензија 3×5 , токен σ_s означен са S:

	Кол. 1	Кол. 2	Кол. 3	Кол. 4	Кол. 5
Ред 0:	A	S	B	D	A
Ред 1:	S	B	A	S	C
Ред 2:	A	S	D	B	A

Укупно 4 појављивања токена S. Ако је табела излазних вредности $\psi_s(3) = 5$, $\psi_s(4) = 20$, $\psi_s(5) = 50$, резултат је 20.

Није битно да су токени S у било каквом реду – битан је само њихов укупан број.

2.11. Активациони токен

Активациони токен σ_b је специјалан токен чије појављивање у довољном броју пута унутар видљивог прозора не даје директан нумеричку излазну вредност, већ мења режим рада система.

Активација наступа када:

$$N_b(W) \geq N_b^{\min}$$

где је $N_b(W)$ број појављивања токена σ_b у прозору, а $N_b^{\min}$ је минимални праг.

Активација може произвести различите ефекте, као на пример:

- Додатне итерације система – систем генерише K додатних итерација које додају излазну вредност на оригиналну итерације, чиме јој се повећава укупна излазна вредност.
- Алтернативна евалуациона функција – привремено се користи другачија табела излазних функција (обично са већим вредностима).
- Модификована генеративна матрица – привремено се користи матрица са другачијом дистрибуцијом токена.

Слично позиционо-инваријантном токenu, и овде се евалуација врши над целим прозором, не дуж путање. Разлика је у ефекту: позиционо-инваријантни токен доприноси излазној вредности у текућој итерацији, док активациони токен покреће нови циклус генерисања и евалуације.

У овом раду неће бити обрађиване могућности овог токена и како он утиче на цео систем. Дефинисан је јер је чест у реалним системима.

2.12. Евалуациона функција и табела излазних вредности итерације

Евалуациона функција је пресликавање које свакој евалуационој секвенци даје нумеричку излазну вредност:

$$\phi: \Sigma^n \rightarrow \mathbb{R}_{\geq 0}$$

Ако евалуациона секвенца садржи образац који систем препознаје као валидан, функција враћа позитивну вредност. У супротном враћа 0.

У најопштијем облику ϕ би могла бити дефинисана као табела са k^n уноса (свака могућа комбинација токена заједно са вредношћу те комбинације). Међутим, то би за скуп од 10 токена и 5 колона захтевало 10^5 уноса. Уместо тога, евалуациона функција се у праксе декомпонује по принципу префикса.

Евалуација по префиксу:

Најчешћи модел евалуације функционише тако што се евалуациона секвенца чита с лева на десно и тражи се најдужи непрекидни низ истог токена на почетку секвенце:

$$\ell(\sigma, e) = \max \{l : e_1 = e_2 = \dots = e_l = \sigma\}$$

За сваки токен σ и дужину префикса l , парцијална функција излазних вредности $f(\sigma, l)$ дефинише одговарајућу нумеричку вредност.

Пример: За скуп токена $\Sigma = \{A, B, C, D\}$ и $n = 5$ колона, евалуациона табела може бити:

Токен	3 узастопна	4 узастопна	5 узастопна
A	5	20	100
B	3	10	50
C	2	5	20
D	1	3	10

На основу ове табеле секвенца (A,A,A,B,C) има префикс од 3 токена A па би излазна вредност функције била: $\phi(e) = 5$.

Својства функције излазних вредности

Функција $f(\sigma, l)$ у пракси увек задовољава:

- Монотоност – већи број узастопних поклапања даје једнаку или већу вредност: $f(\sigma, l_1) \leq f(\sigma, l_2)$ за $l_1 < l_2$
- Минимални праг – постоји минимални број поклапања l_{min} (обично 2 или 3) испод којег је излазна вредност нула.
- Хијерархија токена – различити токени имају различите излазне вредности за исту дужину поклапања. Функција $f(\sigma, l)$ није униформна по σ – неки токени носе веће вредности од других.

2.13. Укупна излазна вредност једне итерације система

Укупна излазна вредност једне итерације је збир доприноса свих компоненти евалуације: евалуација путања „обичних“ токена, број позиционо-инваријантних токена и вредност која произилази из активације активационог токена и његовог секундарног режима.

2.14. Улазна вредност итерације

Свака итерација система захтева дефинисану улазну вредност $c > 0$. Ова вредност је параметар који се задаје пре покретања итерације и не мења се током њеног извршавања.

Излазне вредности које враћа евалуациона функција ϕ и функција излазни вредности $f(\sigma, l)$ су множиоци. Конкретна **излазна вредност система** добија се множењем укупног резултата излазне вредности итерације са улазном вредношћу.

На пример, ако је улазна вредност $c = 2$ и евалуациона функција на некој путањи врати вредност 5, коначна излазна вредност те итерације је $2 \times 5 = 10$.

Улазна вредност не утиче на структуру евалуације нити на вероватноће појављивања токена – она само скалира коначни резултат. Два система са идентичном генеративном матрицом и различитим улазним вредностима имаће исти улазно-излазни однос система.

У даљем тексту, ради једноставности, подразумеваћемо да је улазна вредност $c = 1$. Под овом претпоставком, излазна вредност система је директно једнака излазној вредности итерације, а улазно-излазни однос се своди на $\rho = E[V]$. Сви резултати и закључци важе и за произвољно c јер улазна вредност само скалира коначан резултат без утицаја на структуру система.

2.15. Улазно-излазни однос система

Улазно-излазни однос система је теоријска величина која изражава однос очекиване излазне вредности по итерацији и улазне вредности итерације:

$$\rho = \frac{E[V]}{c}$$

где је $E[V]$ очекивана излазна вредност по итерацији, а $c > 0$ фиксна улазна вредност једне итерације.

Улазно-излазни однос ρ је теоријска вредност која одговара граничној вредности односа укупних улазних и излазних вредности када број итерација тежи бесконачности. Формално, ако систем изврши N итерација са излазним вредностима $V_1, V_2, \dots, V_N$ и улазном вредношћу c по итерацији, тада:

$$\rho = \lim_{N \rightarrow \infty} \frac{\sum_{t=1}^N V_t}{N * c}$$

За коначан број итерација, емпиријски однос може одступати од ρ – након 100 итерација одступање може бити значајно, након 1 000 000 итерација биће занемарљиво. Али само ρ није процена ни апроксимација – то је тачна вредност ка којој систем конвергира, и она је у потпуности одређена структуром генеративне матрице и евалуационим функцијама.

2.16. Варијанса система

Варијанса мери колико се индивидуални излази система у једној итерацији разликују од просека.

Два система могу имати исти коефицијент улазно-излазног односа али потпуно различиту варијансу. Систем са ниском варијансом даје уједначеније резултате – већина итерација има излазну вредност близу просека. Систем са високом варијансом даје већину итерација са излазном вредношћу нула или близу нула, али ретке итерације са изразито високом излазном вредношћу.

Варијанса је важна из два разлога:

1. За конструкцију (Поглавље 3) – варијанса је додатни параметар који се може контролисати подешавањем генеративне матрице.
2. За реконструкцију (Поглавље 4) – већа варијанса значи спорију конвергенцију емпиријских процена ка правим вредностима, односно потребно је више посматрања за исту прецизност реконструкције.

3. Конструкција генеративне матрице са контролисаним улазно-излазним односом

3.1. Увод и формулација инверзног проблема

Када је генеративна матрица позната, израчунавање улазно-излазног односа ρ представља директан проблем: из познатих густина токена и евалуационих правила одређује се очекивана излазна вредност. У овом поглављу разматра се супротан, инверзни проблем — за задату евалуациону табелу и циљану вредност ρ^* потребно је конструисати генеративну матрицу M тако да остварена вредност буде што ближа задатој:

$$\rho(M) \approx \rho^*$$

Проблем није сведен на једноставно решавање једне једначине, већ се ради о n линеарних једначина и једној једначини трећег степена. Овај систем једначина има бесконачно много решења и циљ овог алгоритма је да нађе једно довољно добро. Генеративна матрица је дискретна структура: свака циклична токен-секвенца мора имати целобројну дужину и цео број појављивања сваког токена. Насупрот томе, оптимизациони алгоритми природно раде са реалним вероватноћама. Зато се конструкција спроводи у више фаза:

1. Најпре се у континуалном простору проналазе погодне густине стандардних токена (оптимизација).
2. Затим се те густине претварају у целобројне бројеве појављивања (дискретизација).
3. После додавања специјалних токена поново се израчунава тачан улазно-излазни однос (финална евалуација).
4. На крају се генеришу конкретне цикличне токен-секвенце по добијеним бројевима појављивања за сваки токен.

Овај приступ повезује појмове из теорије вероватноће, нелинеарне оптимизације и структура података. Са становишта рачунарских наука, генеративна матрица је колекција циркуларних низова, док је поступак њене конструкције комбинација оптимизације, пропорционалне дискретизације, исцрпне еnumerације и динамичког програмирања.

Због лакше имплементације математичких функција потребних за оптимизацију и рад са матрицама, за овај део система коришћен је програмски језик Python са библиотекама NumPy и SciPy.

3.2. Густина токена, улаз и излаз конструкције

Као што смо већ дефинисали, нека је $\Sigma_{st} = \{\sigma_1, \sigma_2, \dots, \sigma_k\}$ азбука стандардних токена. Поред њих, систем може садржати универзални заменски токен σ_w и позиционо-инваријантан токен σ_s . Генеративна матрица је уређена n -торка $M = (r_1, r_2, \dots, r_n)$, где је r_j циклична токен-секвенца која одговара j -тој колони. Њена финална дужина означава се са L_j . Ако се токен σ појављује $C_j(\sigma)$ пута у секвенци r_j , његова густина у тој колони је:

$$p_j(\sigma) = \frac{C_j(\sigma)}{L_j}$$

Због униформног избора почетног индекса, свака позиција у цикличној секвенци има једнаку вероватноћу да буде прочитана. Зато је вероватноћа да се на једној позицији видљивог прозора у колони j појави токен σ управо $p_j(\sigma)$.

Улазни подаци за конструкцију су:

- број колона n и висина видљивог прозора m
- азбука стандардних токена Σ_{st}
- евалуациона табела $f(\sigma, l)$
- циљани улазно-излазни однос ρ^*
- бројеви заменских токена w_j по колонама
- бројеви позиционо-инваријантних токена d_j по колонама
- евалуациона табела заменског токена f_w и функција ψ_s за позиционо-инваријантан токен

Резултат је генеративна матрица M , целобројни бројеви појављивања $C_j(\sigma)$, остварени улазно-излазни однос $\rho(M)$ и апсолутно одступање од циљаног односа $\Delta = |\rho(M) - \rho^*|$.

3.3. Очекивана излазна вредност стандардних токена

Основна веза између густина токена и улазно-излазног односа добија се анализом префикса евалуационе секвенце. Нека је $p_j(\sigma_i)$ вероватноћа појављивања токена σ_i у j -тој колони. Због претпостављене независности избора токена по колонама, вероватноћа да првих l елемената евалуационе секвенце буду једнаки токenu σ_i износи:

$$A_{i,l} = \prod_{j=1}^l p_j(\sigma_i)$$

Ово је вероватноћа префикса дужине најмање l . За евалуацију је потребна вероватноћа да је максимална дужина префикса тачно l . Ако је $l < n$, првих l позиција морају садржати σ_i , док позиција $l+1$ мора садржати други токен:

$$P(l_i = l) = \prod_{j=1}^l p_j(\sigma_i) \cdot (1 - p_{l+1}(\sigma_i)), \quad l < n$$

За $l = n$ не постоји наредна колона која би прекинула префикс:

$$P(l_i = n) = \prod_{j=1}^n p_j(\sigma_i)$$

Очекивана излазна вредност система, када су присутни само стандардни токени, једнака је:

$$\rho_{st}(P) = \sum_{i=1}^k \sum_{l=1}^n f(\sigma_i, l) \cdot P(l_i = l)$$

Формула се чита овако: за сваки стандардни токен σ_i (спољашња сума по i) и за сваку могућу дужину поклапања l од 1 до n (унутрашња сума по l), множимо излазну вредност из евалуационе табеле $f(\sigma_i, l)$ са вероватноћом да се то поклапање заиста деси $P(l_i = l)$. Збир свих тих производа даје очекивану излазну вредност система. Суштински, пролазимо кроз све могуће начине на које систем може произвести позитивну излазну вредност (сваки токен $\times$ свака дужина) и за сваки рачунамо колико доприноси просеку — а тај допринос је управо вредност помножена вероватноћом да се оствари.

Формула захтева $O(n \cdot k)$ операција — није потребно пролазити кроз свих k^n могућих евалуационих секвенци. Управо зато се овај израз користи унутар оптимизационог алгоритма, који циљну функцију позива велики број пута.

Пример. Нека за токен A важи $(p_1(A), p_2(A), p_3(A), p_4(A), p_5(A)) = (0.15, 0.12, 0.10, 0.14, 0.11)$ и нека су $f(A, 3) = 5$, $f(A, 4) = 20$ и $f(A, 5) = 100$. Тада је:

$$P(l_A = 3) = 0.15 \times 0.12 \times 0.10 \times (1 - 0.14) = 0.001548$$

$$P(l_A = 4) = 0.15 \times 0.12 \times 0.10 \times 0.14 \times (1 - 0.11) \approx 0.000224$$

$$P(l_A = 5) = 0.15 \times 0.12 \times 0.10 \times 0.14 \times 0.11 \approx 0.0000277$$

$$\text{Допринос токена } A \approx 5 \times 0.001548 + 20 \times 0.000224 + 100 \times 0.0000277 \approx 0.01499$$

Исти поступак се примењује на све остале стандардне токене, а њихови доприноси се сабирају.

Имплементација (Python)

```
def calculate_line_rtp(self, symbol_probabilities, paytable):
    reel_count, normal_symbol_count = symbol_probabilities.shape
    expected_payout = 0.0

    for symbol_id in range(normal_symbol_count):
        prefix_prob = 1.0
        for reel_index in range(reel_count):
            prefix_prob *= symbol_probabilities[reel_index, symbol_id]
            occurrence_count = reel_index + 1

            if occurrence_count == reel_count:
                exact_prob = prefix_prob
            else:
                exact_prob = prefix_prob * (
                    1 - symbol_probabilities[occurrence_count, symbol_id]
                )
            expected_payout += paytable[symbol_id, occurrence_count] * exact_prob

    return expected_payout
```

3.4. Оптимизација густине токена

Променљиве и ограничења

Свака вероватноћа мора бити не негативна (не може бити негативна вероватноћа појављивања токена). Поред тога, за сваку колону j , збир вероватноћа свих стандардних токена мора бити тачно 1. На пример, ако у колони 0 имамо три токена са вероватноћама 0.45, 0.35 и 0.20, њихов збир је 1.00. Оптимизатор не сме произвести решење где би тај збир био, рецимо, 0.87 или 1.15. То се математички записује:

$$p_{j,i} \geq 0, \quad \sum_{i=1}^k p_{j,i} = 1, \quad j = 1, \dots, n$$

Укупно има $n \times k$ променљивих.

Улаз у оптимизацију су циљани улазно-излазни однос ρ^* и евалуациона табела $f(\sigma, l)$. Излаз су оптималне вероватноће $p_j(\sigma_i)$ за сваку колону и сваки стандардни токен. Важно је нагласити да се оптимизација врши искључиво над стандардним токенима — заменски и позиционо-инваријантни токени нису укључени у овом кораку. Њихов допринос се израчунава касније, након дискретизације.

Поред тога променљиве p_{ij} , $i = 1, k$, $j=1, n$ морају да задовољавају једначину улазно-излазног односа:

$$\sum_{i=1}^k \sum_{l=1}^n f(\sigma_i, l) \cdot P(l_i = l) = \rho^*$$

Систем једначина решавамо нумерички како би смо дошли до једног задовољавајућег решења, тако што минимизирамо функцију циља као квадрат одступања од циљане вредности:

$$J(P) = (\rho_{st}(P) - \rho^*)^2$$

За решавање овог проблема користи се SLSQP (Sequential Least Squares Programming) — метода нелинеарног програмирања која омогућава истовремено задавање граница променљивих са ограничењем у виду једнакости.

Избор почетних тачака

SLSQP је метода локалне оптимизације — гарантује конвергенцију ка локалном минимуму, али не нужно ка глобалном. Различита почетна решења могу водити ка различитим локалним минимумима, од којих неки могу значајно одступати од циљаног ρ^* . Зато се оптимизација покреће из више различитих почетних тачака, а на крају се бира најбољи резултат.

Почетне тачке се генеришу различитим стратегијама:

- Геометријске расподела — токенима се додељују тежине α^i и нормализују. Различите вредности α и различити помаци (stride) дају расподеле различитог степена неравномерности.
- Расподела засноване на евалуационој табели — токени са већим излазним вредностима добијају мање или веће почетне густине, зависно од жељеног подручја претраге.
- Дирихлеове расподеле — сваки ред матрице случајно се узоркује из Дирихлеове расподеле, чиме се добијају допустиве почетне тачке без накнадне нормализације.

Кључан детаљ је да се кандидати не пореде само у континуалном простору. Кандидат који даје готово савршену вредност $\rho_{st}(P)$ може бити лош након претварања у целобројне бројеве појављивања. Зато се сваки конвергирани кандидат привремено дискретизује, израчунава се однос добијен дискретним густинама и тек онда се одређује његов квалитет. Бира се кандидат чија дискретизована вредност најмање одступа од ρ^* .

Имплементација (Python)

```
def optimize(self, reel_count, paytable, target_rtp, payrule, score_fn=None):
    normal_symbol_count = paytable.shape[0]
    initial_candidates = self._build_initial_probability_candidates(
        reel_count, normal_symbol_count, paytable
    )

    constraints = [
        {
            'type': 'eq',
            'fun': lambda probs, r=r: np.sum(
                probs[r * normal_symbol_count:(r + 1) * normal_symbol_count]
            ) - 1
        }
        for r in range(reel_count)
    ]

    def objective(flat_probs):
        probs_matrix = flat_probs.reshape(reel_count, normal_symbol_count)
        rtp = payrule.calculate_line_rtp(probs_matrix, paytable)
        return (rtp - target_rtp) ** 2

    best_probabilities = None
    best_error = float('inf')

    for initial in initial_candidates:
        result = minimize(
            fun=objective, x0=initial.ravel(), method='SLSQP',
            bounds=[(0, 1)] * (reel_count * normal_symbol_count),
            constraints=constraints,
        )
        if not result.success:
            continue

        optimized = result.x.reshape(reel_count, normal_symbol_count)

        score_rtp = score_fn(optimized) if score_fn else \
            payrule.calculate_line_rtp(optimized, paytable)
        error = abs(score_rtp - target_rtp)

        if error < best_error:
            best_probabilities = optimized
            best_error = error

    return best_probabilities
```

3.5. Дискретизација

Зашто је дискретизација неопходна

Оптимизација може дати, на пример, $p_j(\sigma_i) = 0.3333\dots$ Циклична токен-секвенца, међутим, не може садржати 33.33 копија токена (за дужину 100). За сваку колону мора се пронаћи целобројни вектор $(C_j(\sigma_1), \dots, C_j(\sigma_k))$ чији је збир једнак дужини секвенце и чије дискретне густине $C_j(\sigma)/L_j$ што боље апроксимирају оптимизоване вероватноће.

Метода највећег апсолутног остатка

Проблем претварања континуалних вероватноћа у целобројне бројеве појављивања има бесконачно много решења — за сваку могућу дужину секвенце постоји одговарајућа целобројна додела. Секвенца дужине 100, 157 или 843 може садржати различите бројеве токена који сви апроксимирају исте вероватноће, али са различитим квалитетом апроксимације. Зато се дефинише опсег допустивих дужина $[L_{\min}, L_{\max}]$ и алгоритам испробава сваку целобројну дужину у том опсегу, тражећи ону која даје најмању грешку.

За сваку дужину L из опсега, поступак је следећи: ако желимо да токен σ_i заузима удео p_i секвенце, идеалан број његових појављивања је $e_i = L \cdot p_i$. Овај број скоро никад није цео — на пример, за $L = 97$ и $p = 0.45$ добијамо $e = 43.65$. Зато сваком токenu најпре доделимо цео део тог броја: 43.65 постаје 43. Тиме укупан збир свих додела буде мањи од L — преостане R нераспоређених позиција. Те позиције додељујемо једну по једну токенима којима је заокруживањем одузето највише (они са највећим децималним остатком), чиме се грешка расподељује што правичније.

Овај поступак се понови за сваку дужину од $L_{\min}$ до $L_{\max}$. За сваку добијемо целобројну доделу и израчунамо максималну апсолутну грешку — колико се дискретне густине разликују од циљаних вероватноћа. На крају бирамо дужину која даје најмању грешку. Тиме из бесконачног скупа могућих решења бирамо једно конкретно које најбоље чува оригиналне вероватноће у задатом опсегу.

Квалитет апроксимације за дату дужину мери се максималном апсолутном грешком:

$$\delta_j(L) = \operatorname{argmax}_{1 \leq i \leq k} \left| p_j(\sigma_i) - \frac{C_j(\sigma_i)}{L} \right|$$

Имплементација (Python)

```
def discretize_single_reel(self, target_probabilities):
    target = np.asarray(target_probabilities, dtype=float)
    target = target / target.sum()

    best_error = float('inf')
    best_length, best_counts = 0, None

    for length in range(self.min_reel_length, self.max_reel_length + 1):
        exact = length * target
        counts = np.floor(exact).astype(int)
        remaining = int(length - counts.sum())
        remainders = exact - counts
        top_indices = np.argsort(remainders)[::-1]
        counts[top_indices[:remaining]] += 1
        probs = counts / length
        error = np.abs(target - probs).max()

        if error < best_error:
            best_error = error
            best_length = length
            best_counts = counts

    return ReelConfiguration(
        length=best_length, symbol_counts=best_counts,
        symbol_probabilities=best_counts / best_length,
        max_absolute_error=best_error,
    )
```

3.6. Укључивање специјалних токена

Након дискретизације стандардних токена додају се специјални токени. Њихов број по колони је задат улазним параметром. Додавањем нових токена дужина секвенце се повећава:

$$L_j^{fin} = L_j^{st} + w_j + d_j$$

где је L_j^{st} дужина након дискретизације стандардних токена, w_j број заменских токена, а d_j број позиционо-инваријантних токена у колони j .

Ово повећање дужине мења густине свих токена јер је именилац у формули за густину већи:

$$p_j^{fin}(\sigma_i) = \frac{C_j(\sigma_i)}{L_j^{fin}}$$

На пример, ако је $C_j(A) = 20$ и $L_j^{st} = 150$, стара густина је $20/150 = 0.1333$. Након додавања $w_j = 1$ заменског и $d_j = 4$ позиционо-инваријантна токена, нова дужина је 155, а нова густина стандардног токена А је $20/155 = 0.1290$.

Густине специјалних токена су:

$$p_j(\sigma_w) = \frac{w_j}{L_j^{fin}}, \quad p_j(\sigma_s) = \frac{d_j}{L_j^{fin}}$$

Због промене свих густина, континуални резултат из одељка 3.4 више није коначна вредност. Неопходно је поновно израчунавање целокупног односа са финалним густинама.

Универзални заменски токен

Када систем садржи универзални заменски токен, формула префикса из Секције 3.3 више није довољна. Та формула посматра сваки токен независно — рачуна вероватноћу префикса за токен А, па за токен В, и тако даље, и сабере доприносе. Али заменски токен компликује ствари јер он може „глумити“ било који стандардни токен. Секвенца (А, σ_w , А, В, С) може се евалуирати као да је σ_w заправо А — чиме се добија префикс дужине 3. Тај допринос се не може приписати ни самом токenu А (јер није чист А низ) ни самом заменском токenu (јер он нема фиксну улогу), већ зависи од комбинације у којој се нашао.

Зато се користи директнији приступ: пролази се кроз све могуће комбинације токена које се могу појавити на евалуационој секвенци. Сада се не разматрају само стандардни токени, већ и заменски — укупно $k+1$ различитих типова токена. За секвенцу дужине n , број свих могућих комбинација је $(k+1)^n$. За сваку комбинацију израчуна се њена вероватноћа и излазна вредност, и сабере на укупну суму. Овај приступ је спорији од формуле префикса, али даје тачан резултат јер узима у обзир све могуће интеракције заменског токена са стандардним токенима.

За сваку комбинацију $t = (t_1, \dots, t_n) \in \Sigma_w^n$:

1. Вероватноћа комбинације — производ финалних густина по колонама:

$$P(t) = \prod_{j=1}^n p_j^{fin}(t_j)$$

2. Евалуација — ако комбинација садржи σ_w , испробавају се све могуће замене стандардним токенима и бира се она са највећом излазном вредношћу.
3. Допринос заменског токена — поред најбоље замене, заменски токен има и сопствену евалуациону табелу $f_w(l)$ која дефинише додатну излазну вредност на основу броја његових појављивања N_w у комбинацији. Укупна вредност комбинације је:

$$\phi_w(t) = \phi_{best}(t) + f_w(N_w(t))$$

4. Акумуляција — укупна излазна вредност множи се са вероватноћом и додаје на суму:

$$\rho_w = \sum_{t \in \Sigma_w^n} P(t) \cdot \phi_w(t)$$

Ова сума већ садржи и комбинације без заменског токена (чисте стандардне комбинације), па ρ_w у потпуности замењује ρ_{st} — није потребно додатно сабирање.

Важно је разумети зашто се енумерација не користи током оптимизације (Секција 3.4), већ тек у овом кораку: оптимизациони алгоритам позива функцију за израчунавање очекиване вредности стотине пута током конвергенције. Формула префикса из Секције 3.3 има $k \times n$ операција (нпр. 60 за $k = 12$, $n = 5$). Исцрпна енумерација има $(k+1)^n$ комбинација (нпр. 371 293 за $k = 12$, $n = 5$) и била би превише спора за поновљено позивање. Зато се енумерација примењује само једном, на крају, са већ познатим бројевима токена.

```
def calculate(self, normal_symbol_probabilities, paytable, wild_paytable,
              wild_symbol_counts, reel_lengths, payrule):
    reel_count, normal_symbol_count = normal_symbol_probabilities.shape
    wild_symbol_id = normal_symbol_count
    total_symbol_count = normal_symbol_count + 1
    wild_probabilities = wild_symbol_counts / reel_lengths

    expected_payout = 0.0
    for combination in product(range(total_symbol_count), repeat=reel_count):
        prob = self._combination_probability(
            combination, normal_symbol_probabilities,
            wild_probabilities, wild_symbol_id
        )
        payout = self._combination_payout(
            combination, paytable, wild_paytable, wild_symbol_id, payrule
        )
        expected_payout += prob * payout

    return expected_payout
```

Позиционо-инваријантни токен

Позиционо-инваријантан токен се не евалуира дуж путања као стандардни токени, него се броји колико пута се укупно појављује у целом видљивом прозору. За то нам треба вероватноћа да се уопште појави у свакој колони.

Видљиви прозор из сваке колоне приказује m узастопних позиција из цикличне секвенце дужине L_j^{fin} . Ако се позиционо-инваријантан токен појављује d_j пута у тој секвенци, свако

појављивање „покрива“ m могућих почетних индекса који би га учинили видљивим (јер прозор чита m узастопних позиција). Укупно $m \cdot d_j$ почетних индекса од L_j^{fin} могућих ће резултирати тиме да се бар један позиционо-инваријантан токен нађе у видљивом делу колоне. Зато је вероватноћа:

$$q_j = \frac{m \cdot d_j}{L_j^{\text{fin}}}$$

Ова апроксимација важи под претпоставком да је густина позиционо-инваријантног токена релативно мала, тако да је вероватноћа његовог вишеструког појављивања у истој колони видљивог прозора једнака 0.

Укупан број појављивања N_s у прозору је сума независних Бернулијевих случајних променљивих — по једна за сваку колону, где свака има своју вероватноћу q_j . Пошто вероватноће нису нужно једнаке, N_s има Поасон-биномна расподелу, са законом расподеле:

$$a_r^{(j)} = a_r^{(j-1)}(1 - q_j) + a_{r-1}^{(j-1)} q_j$$

Након свих n колона важи $a_r^n = P(N_s = r)$. Очекивани допринос позиционо-инваријантног токена је скаларни производ вектора вредности и вектора вероватноћа:

$$\rho_s = \sum_{r=0}^n \psi_s(r) \cdot P(N_s = r)$$

Имплементација (Python)

```
def calculate_occurrence_count_probabilities(observation_probabilities):
    count_probabilities = np.array([1.0])
    for observation_probability in observation_probabilities:
        updated = np.zeros(len(count_probabilities) + 1)
        for k in range(len(count_probabilities)):
            updated[k] += count_probabilities[k] * (1 - observation_probability)
            updated[k + 1] += count_probabilities[k] * observation_probability
        count_probabilities = updated
    return count_probabilities

def calculate(self, visible_row_count, scatter_paytable,
              scatter_symbol_counts, reel_lengths):
    visibility_probabilities = (
        visible_row_count * scatter_symbol_counts / reel_lengths
    )
    count_probabilities = calculate_occurrence_count_probabilities(
        visibility_probabilities
    )
    return float(np.dot(scatter_paytable, count_probabilities))
```

3.7. Укупни улазно-излазни однос и финалне провере

Када систем садржи универзални заменски токен и позиционо-инваријантан токен, укупни улазно-излазни однос је:

$$\rho(M) = \rho_w(M) + \rho_s(M)$$

где ρ_w већ укључује стандардне токене. Ако заменски токен није присутан, ρ_w се замењује са ρ_{st} .

3.8. Генерисање цикличне токен-секвенце

Када су бројеви појављивања коначно утврђени, за сваку колону се формира мултикуп токена. У низ се дода $C_j(\sigma)$ копија сваког токена, након чега се низ псеудо-случајно пермутује. Коришћење познате почетне вредности генератора обезбеђује репродуцибилност.

Имплементација (Python)

```
def generate_strip(self, symbol_counts, random):
    pieces = []
    for symbol_id, count in enumerate(symbol_counts):
        if count > 0:
            pieces.append(np.full(count, symbol_id, dtype=int))

    strip = np.concatenate(pieces)
    random.shuffle(strip)
    return strip
```

Утицај распореда токена. Редослед токена не мења густине из формуле за $p_j(\sigma)$ и зато не мења улазно-излазни однос ρ . Међутим, редослед утиче на зависност између суседних редова видљивог прозора, а самим тим и на варијансу излазне вредности.

Груписање истих токена повећава вероватноћу да се они истовремено појаве на више блиских позиција, што повећава варијансу. Равномернији распоред смањује ту локалну зависност. За позиционо-инваријантан токен, редослед може утицати и на средњу вредност ако није задовољена претпоставка ретког и раздвојеног распореда. У том случају, токене σ_s треба поставити уз минимално циклично растојање од најмање m позиција.

Анализа утицаја распореда на варијансу и методе оптималног позиционирања токена превазилазе оквир овог рада. У имплементацији је коришћено псеудо-случајни генератор за пермутовање.

3.9. Закључак поглавља

Конструкција генеративне матрице са задатим улазно-излазним односом представља инверзни проблем у којем се од жељене статистичке особине долази до конкретне дискретне структуре података. Главна тешкоћа је разлика између континуалног простора вероватноћа, погодног за нелинеарну оптимизацију, и целобројне природе цикличних токен-секвенци.

Поступак зато комбинује више алгоритамских техника. Аналитичка формула за очекивану вредност префиксне евалуације омогућава брзо израчунавање унутар SLSQP оптимизације. Метода највећег остатка претвара добијене густине у целобројне бројеве појављивања, уз претрагу погодних дужина секвенци. Исцрпна еnumerација обезбеђује прецизну обраду универзалног заменског токена, док динамичко програмирање ефикасно израчунава Поасон-биномна расподелу броја позиционо-инваријантних токена.

Коначни резултат није само матрица вероватноћа, већ потпуно одређена генеративна матрица: за сваку колону позната је дужина, број појављивања сваког токена и конкретан циклични распоред. Теоријска вредност мора се израчунати над том коначном структуром и проверити симулацијом.

4. Реконструкција цикличне токен-секвенце помоћу Де Брујнових графова

4.1. Уводне напомене и формулација проблема

У Поглављу 3 описан је поступак конструкције генеративне матрице — од задатог улазно-излазног односа p^* до конкретних цикличних токен-секвенци. У овом поглављу разматрамо супротан проблем: ако нам генеративна матрица није позната, да ли можемо реконструисати оригиналне токен-секвенце из великог броја посматраних итерација?

На располагању имамо велики скуп података прикупљен посматрањем система (подаци посматрања) — сваки запис представља видљиви прозор димензија $m \times n$ добијен у једној итерацији. Не знамо како изгледају цикличне токен-секвенце из тих прозора, не знамо њихове дужине, нити знамо колико пута се који токен појављује. Знамо само оно што нам прозор показује: m узастопних токена из сваке колоне.

Кључна опсервација је да се свака колона може реконструисати независно. Видљиви прозор приказује m узастопних токена из сваке цикличне секвенце, и ти токени долазе искључиво из те секвенце — нема интеракције између колона у фази генерисања. Зато се проблем реконструкције целе генеративне матрице своди на n независних проблема реконструкције појединачних цикличних токен-секвенци.

За сваку колону, из великог броја посматраних итерација имамо колекцију кратких фрагмената дужине m — узастопних подсеквенци оригиналне цикличне секвенце. Питање је: како из тих фрагмената саставити комплетну секвенцу?

4.2. Аналогија са биоинформатиком

Овај проблем има изненађујуће блиску аналогију у потпуно другој области — биоинформатици. Савремене технологије за секвенцирање ДНК имају физичка ограничења због којих не могу да читају читаве, изузетно дугачке молекуле одједном. Да би се то превазишло, у лабораторији се прво прави огроман број идентичних копија циљне ДНК, а затим се оне насумично ломе (фрагментишу) на милионе кратких делова. Пошто се различите копије ланца ломе на различитим местима, добијени фрагменти — које машине читавају као кратке секвенце (енгл. reads) — међусобно се преклапају. Задатак биоинформатичара је да, без директног увида у комплетан ланац, искористе ова преклапања како би склопили ове фрагменте назад у једну непрекидну, оригиналну ДНК секвенцу.

Ситуација је готово идентична нашој: не видимо целу цикличну токен-секвенцу, видимо само кратке фрагменте (прозоре висине m) и морамо да их склопимо. Штавише, и у биоинформатици и у нашем проблему, иста подсеквенца може се појавити на више места у оригиналу, што компликује реконструкцију.

Алгоритамски приступ који је постао стандард у биоинформатици за овај тип проблема заснива се на Де Брујновим графовима — структури коју је описао холандски математичар Николас Говерт де Брујн 1946. године. Исти приступ можемо директно применити на реконструкцију наших цикличних секвенци.

4.3. Дефиниција модела

Да бисмо изградили Де Брујнов граф, потребно је најпре дефинисати основне појмове.

k-мери

Видљиви прозор висине m из једне колоне даје нам тачно m узастопних токена из цикличне секвенце. Овај уређени подниз дужине m називамо k -мер, где је $k = m$. За систем са прозором висине $m = 3$, сваки посматрани фрагмент је триграм — уређена тројка узастопних токена.

На пример, ако прозор у једној итерацији прикаже токене $[A, B, C]$ одозго надоле у некој колони, то значи да негде у цикличној секвенци постоји подниз $\dots A, B, C \dots$ где A претходи B -у, а B претходи C -у.

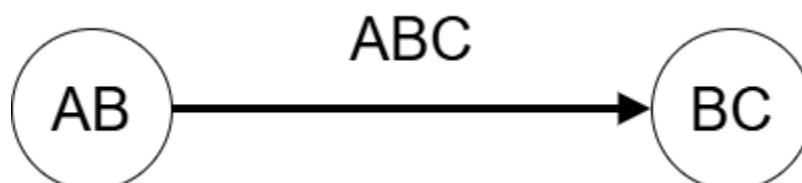
Чворови и ивице

Де Брујнов граф се конструише тако што се k -мери „разлажу” на преклапајуће делове:

- Чворови графа представљају подсеквенце дужине $k-1$ (за триграме су то биграми — парови узастопних токена).
- Усмерене ивице (гране) представљају посматране k -мере.

Конкретно, триграм $[A, B, C]$ се разлаже на:

- улазни чвор: (A, B) — прва два токена (префикс)
- излазни чвор: (B, C) — последња два токена (суфикс)
- ивицу од (A, B) ка (B, C) , означену триграмом ABC



Разлог за овакву конструкцију је преклапање. Ако у нашем скупу података имамо два триграма $[A, B, C]$ и $[B, C, D]$, они деле биграм (B, C) — крај првог триграма је почетак другог. У графу, оба триграма деле чвор (B, C) : први триграм улази у тај чвор, а други излази из њега. Тиме се граф природно повезује и омогућава нам да пратимо ланац преклапања кроз целу секвенцу.

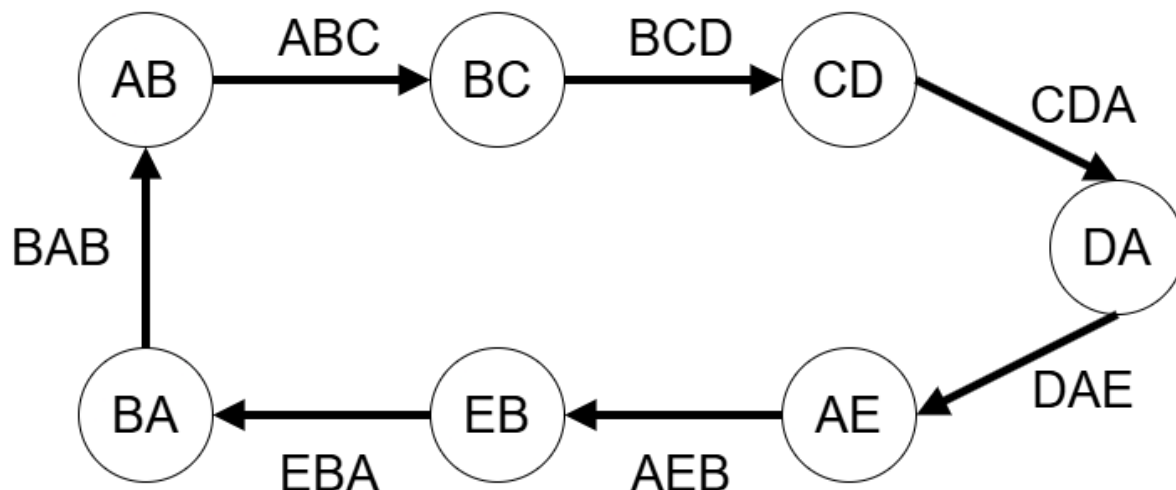
4.4. Изградња графа из посматраних узорака

За једну колону, поступак изградње графа је следећи:

1. Из сваке посматране итерације извучемо k -мер (триграм уколико је $m = 3$ узастопна токена) за ту колону.
2. За сваки k -мер формирамо два чвора, префикс и суфикс, и једну усмерену ивицу.
3. Ако чвор већ постоји у графу, не дуплирамо га — чворови су јединствени.
4. За сваку ивицу бројимо колико пута се одговарајући триграм појавио у скупу података који имамо (ово је битно за решавање проблема дупликата у Секцији 4.6).

Пример. Нека имамо следећих 5 различитих триграма које смо посматрали за једну колону: $[A, B, C]$, $[B, C, D]$, $[C, D, A]$, $[D, A, E]$, $[A, E, B]$, $[E, B, A]$, $[B, A, B]$. Де Брујнов граф садржи:

- Чворове: (A, B) , (B, C) , (C, D) , (D, A) , (A, E) , (E, B) , (B, A)
- Гране: $AB \rightarrow BC$, $BC \rightarrow CD$, $CD \rightarrow DA$, $DA \rightarrow AE$, $AE \rightarrow EB$, $EB \rightarrow BA$, $BA \rightarrow AB$



4.5. Ојлеров циклус и реконструкција секвенце

Кружна природа секвенце

Пошто су цикличне токен-секвенце кружне структуре (након последњег токена следи први), реконструисана секвенца мора формирати затворену петљу. У терминима теорије графова, тражимо затворени пут кроз граф.

Ојлеров циклус

Реконструкција секвенце се своди на проналажење Ојлеровог циклуса у изграђеном Де Брујновом графу. Ојлеров циклус је затворени пут који пролази кроз сваку грану графа тачно једном и враћа се у полазни чвор.

Зашто баш Ојлеров циклус? Свака ивица у нашем графу представља један посматрани к-мер — један стваран фрагмент цикличне секвенце. Проласком кроз сваку ивицу тачно једном, ми заправо уланчавамо све фрагменте у правилан редослед и добијамо комплетну секвенцу.

Зашто не Хамилтонов пут?

Алтернативан приступ био би тражење Хамилтоновог пута — пута који обилази сваки чвор у графу тачно једном. Међутим, концепт Хамилтоновог пута је суштински погрешан за наш проблем из два кључна разлога: структурног и рачунског.

Структурно ограничење Хамилтоновог пута (које забрањује поновну посету већ обиђеном чвору) директно се сукобљава са реалним дизајном цикличних токен-секвенци. Наиме, на реалној секвенци се поједини биграми (нпр. комбинација симбола АВ) могу појавити на више различитих места. То значи да ће наш граф имати чворове кроз које морамо проћи више пута како бисмо успешно искористили све ивице које из њих излазе. Хамилтонов пут би нас приморао да сваки чвор посетимо само једном, што би математички онемогућило реконструкцију било које траке која садржи дуплиране симболе. Насупрот томе, Ојлеров циклус се фокусира на гране (триграме) које морају бити искриштене тачно једном, док саме чворове (биграме) дозвољава и захтева да посетимо онолико пута колико је потребно.

Док са друге стране, Ојлеров циклус се може пронаћи у линеарном времену $O(|E|)$, где је $|E|$ број ивица у графу, коришћењем класичног Хирхолцеровог алгоритма.

Потребан и довољан услов за постојање Ојлеровог пута

Једини услов за постојање Ојлеровог циклуса у усмереном графу јесте да за сваки чвор његов улазни степен (број улазних ивица) буде једнак његовом излазном степену (број излазних ивица):

$$\deg^{in}(v) = \deg^{out}(v) \quad \text{за сваки чвор } v$$

У правилно изграђеном Де Брујновом графу цикличне секвенце, овај услов је увек испуњен: сваки триграм који „улази“ у неки биграма-чвор има одговарајући триграм који из њега „излази“ (јер је секвенца кружна и свака позиција има и претходника и следбеника).

Од Ојлеровог циклуса до секвенце

Када пронађемо Ојлеров циклус, секвенцу добијамо једноставним читањем. Ако је циклус:

$$(A,B) \rightarrow (B,C) \rightarrow (C,D) \rightarrow (D,A) \rightarrow (A,E) \rightarrow (E,B) \rightarrow (B,A) \rightarrow (A,B)$$

Секвенцу формирамо тако што из сваког чвора узмемо само први токен (јер се други токен поклапа са првим токеном наредног чвора): A, B, C, D, A, E, B — и то је наша реконструисана циклична токен-секвенца дужине 7.

Идеалан случај

Ако у цикличној секвенци не постоје поновљени биграми (сваки пар узастопних токена је јединствен), граф има тачно онолико ивица колико је дужина секвенце, сваки чвор има улазни и излазни степен тачно 1, и Ојлеров циклус је јединствен. У том случају реконструкција је тривијална и савршена.

Међутим, у реалним системима токени се понављају, исти пар токена може се јавити на више места у секвенци, и граф добија сложенију структуру. Ојлеров циклус и даље постоји, али може их бити више — граф може имати више валидних циклуса. Чак и у том случају, сви циклуси дају секвенце са истим статистичким својствима (исте густине токена, исте вероватноће триграма), што значи да за потребе израчунавања улазно-излазног односа свака реконструкција даје исправан резултат.

4.6. Проблема дупликата и његово решење

У пракси се суочавамо са фундаменталним проблемом: како разликовати учесталост виђања од физичког присуства токена?

Ако посматрамо систем кроз велики број итерација и забележимо триграм [А, В, С] укупно 10 000 пута, то може значити две ствари:

1. Секвенца [А, В, С] се налази на само једном месту на цикличној секвенци, али се систем као излаз дао ту секвенцу хиљадама пута.
2. Секвенца [А, В, С] физички постоји на више различитих места на секвенци (поновљени подниз).

Ова разлика је критична за исправну реконструкцију. Ако триграм постоји на два места, у Де Брујновом графу морамо имати две паралелне ивице између истих чворова — граф постаје мултиграф.

Статистичко скалирање

Решење се заснива на претпоставци униформности псеудо случајног генератора коју смо увели у Секцији 2.5: свака позиција у цикличној секвенци има једнаку вероватноћу да буде изабрана као почетни индекс. Ако секвенца има дужину N , а ми посматрамо M итерација, вероватноћа појављивања подсеквенце у скупу података:

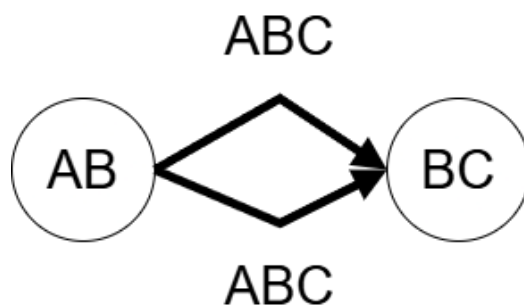
$$F_{base} = \frac{M}{N}$$

Ова вредност представља базну фреквенцију —вероватноћа број појављивања k -мера који се на секвенци налази тачно једном.

Ако се триграм [А, В, С] у подацима посматрања појављује приближно F_{base} пута, закључујемо да на секвенци постоји тачно једна позиција на којој се налази та секвенца токена. Ако се појављује приближно $2 \cdot F_{base}$ пута, закључујемо да постоје две физичке позиције са том секвенцом. Генерално, капацитет (вишеструкост) сваке ивице у графу одређујемо као:

$$\text{капацитет}(e) = \text{round}\left(\frac{\text{број појављивања}(e)}{F_{base}}\right)$$

Ако је капацитет 1, ивица остаје једна. Ако је капацитет 2, уводимо две паралелне ивице између истих чворова — граф постаје мултиграф. Ојлеров циклус у мултиграфу пролази кроз сваку ивицу (укључујући паралелне) тачно једном, чиме се поновљени подниз правилно реконструише на оба места у секвенци.



Пример. Нека је дужина секвенце $N = 50$, а скуп података садржи $M = 1\,000\,000$ итерација. Базна фреквенција је $F_{\text{base}} = 1\,000\,000 / 50 = 20\,000$. Триграм који се појављује приближно 20 000 пута постоји на једном месту; триграм који се појављује приближно 40 000 пута постоји на два места; триграм који се појављује приближно 60 000 пута постоји на три места на секвенци.

Одређивање непознате дужине секвенце

У претходном одељку претпоставили смо да је дужина секвенце N позната. Међутим, у реалном сценарију реконструкције, N нам на почетку није познато — управо покушавамо да реконструишемо секвенцу чију дужину не знамо.

Решење се заснива на анализи дистрибуције фреквенција свих посматраних k -мера. Када за сваки јединствени k -мер пребројимо колико пута се појавио у скупу података и те бројеве сортирамо, приметимо да се они групишу у јасне, дискретне кластере.

Прва група садржи k -мере који се појављују приближно исти (најмањи) број пута — то су k -мери који на секвенци постоје тачно једном. Друга група садржи k -мере са приближно двоструком фреквенцијом — то су k -мери који постоје на два места. Трећа група има троструку фреквенцију, итд.

Најнижа стабилна група фреквенција (означимо је са F_{min}) представља базну фреквенцију — фреквенцију k -мера који се на секвенци појављују тачно једном. Из ње можемо израчунати непознату дужину секвенце:

$$N = \frac{M}{F_{\text{min}}}$$

Пример. Ако скуп података садржи $M = 1\,000\,000$ итерација и најнижа стабилна група k -мера има просечну фреквенцију $F_{\text{min}} \approx 20\,000$, закључујемо да је дужина секвенце $N \approx 1\,000\,000 / 20\,000 = 50$. Сада можемо израчунати капацитете свих ивица и изградити комплетан мултиграф.

Робусност процене

Прецизност ове процене директно зависи од броја посматраних итерација M . Ако је M мали, свака позиција на секвенци биће посећена мали број пута, па ће емпиријске фреквенције значајно варирати око теоријских вредности. Триграм који се на секвенци налази једном можда буде посматран 18 пута, а други триграм који се такође налази једном можда 23 пута — иако би оба требало да имају исту фреквенцију. У таквим условима кластери се преклапају и тешко их је раздвојити.

Како M расте, ситуација се драматично побољшава. Закон великих бројева гарантује да емпиријске фреквенције конвергирају ка теоријским вредностима. За $M = 1\,000\,000$ и секвенцу дужине $N = 50$, триграм који постоји на једном месту имаће фреквенцију веома близу 20 000, а триграм који постоји на два места веома близу 40 000. Размак између ових група (20 000) постаје толико већи од случајних одступања унутар сваке групе да се кластери јасно раздвајају.

У пракси, однос $M/N > 100$ (свака позиција посећена најмање стотинак пута) обезбеђује довољну прецизност да се кластери поуздано идентификују и да се капацитети ивица исправно одреде.

4.7. Ограничења и напомене

Јединственост реконструкције. Ако Де Брујнов граф има више валидних Ојлерових циклуса (што се дешава када постоје чворови са степеном већим од 1), реконструисана секвенца можда неће бити идентична оригиналу. Међутим, све валидне реконструкције имају исте густине токена и исте фреквенције свих подсеквенци, па дају исте вредности при израчунавању улазно-излазног односа. За редослед токена унутар секвенце, који утиче на варијансу (као што је објашњено у Секцији 3.8), реконструкција може дати различит распоред од оригиналног.

Потребан број посматрања. Да би реконструкција била поуздана, потребно је довољно велико M да се сви k -мери који постоје на секвенци појаве бар неколико пута у подацима посматрања. За секвенцу дужине N , минималан број посматрања треба да буде знатно већи од N — у пракси, $M/N > 100$ обезбеђује довољну прецизност за раздвајање кластера фреквенција.

Ретки k -мери. Ако неки k -мер на секвенци постоји само једном, а број итерација није довољно велики, постоји вероватноћа да тај k -мер уопште не буде посматран. У том случају, реконструисана секвенца ће имати „рупу“ — недостајаће ивица у графу, Ојлеров циклус неће постојати и реконструкција неће бити могућа без додатних претпоставки. Ово је додатан разлог зашто је велики скуп података посматрања критичан за успешну реконструкцију.

4.8. Закључак поглавља

Реконструкција цикличне токен-секвенце из посматраних фрагмената елегантно се моделира помоћу Де Брујнових графова. Исти приступ који биоинформатичари користе за склапање ДНК секвенци из милиона кратких читања примењује се на склапање цикличних секвенци из посматраних видљивих прозора.

Кључна предност овог приступа је рачунска ефикасност: проблем реконструкције се своди на проналажење Ојлеровог циклуса, који се решава у линеарном времену. Проблем дупликата — разликовање учесталости виђања од физичког присуства — решава се статистичким скалирањем на основу претпоставке униформности, чиме се граф прецизно претвара у мултиграф са исправним вишеструкостима ивица.

5. Компаративна анализа алгоритма за евалуацију комбинација

5.1. Уводне напомене

У претходним поглављима дефинисан је формални модел система и описана конструкција генеративне матрице. Овим поглављем прелазимо на проблем који се тиче ефикасности самог извршавања система: како за дати видљиви прозор W што брже евалуирате све евалуационе путање и израчунати укупну излазну вредност итерације.

Проблем је на први поглед једноставан – за сваку путању из скупа P треба прочитати одговарајуће токене из прозора и применити евалуациону функцију ϕ . Међутим, када систем треба да изврши милионе или десетине милиона итерација (нпр. приликом симулације за верификацију улазно-излазног односа, или у контексту реконструкције матрице из Поглавља 4, или за сам реалан рад система за пуно корисника), ефикасност евалуације постаје критичан фактор.

У овом поглављу анализирамо три приступа евалуацији:

1. **Секвенцијална евалуација** – свака путања се евалуира независно, проласком кроз све позиције евалуационе путање.
2. **Евалуација помоћу хеш таблице** – све могуће комбинације токена које дају позитивну излазну вредност унапред се израчунају и сместе у хеш табелу, евалуације се своди на претрагу табеле.
3. **Евалуација помоћу индексног стабла** – евалуационе путање се организују у стабло (trie структуру) које омогућава дељење заједничких префикса и рано одсецање грана.

За сваки приступ дајемо опис алгоритма, анализу временске и меморијске сложености, и имплементационе детаље. На крају поглавља показујемо резултате експерименталног поређења сва три приступа.

5.2. Секвенцијална евалуација

Опис приступа

Секвенцијална евалуација је најдиректнији приступ проблему. Свака евалуациона путања се обрађује посебно, једна по једна, од прве до последње.

Да бисмо разумели како овај приступ ради, морамо најпре разјаснити како се евалуационе путање чувају у меморији и како се користе за читање токена из видљивог прозора.

Репрезентација евалуационих путања

Свака евалуациона путања је низ он n целих бројева, где сваки број представља индекс реда у одговарајућој колони видљивог прозора. Ако систем им $n = 5$ и $m = 3$ реда (индекси 0, 1, 2, 3), једна путања изгледа овако:

Путања: [0, 1, 2, 1, 0]

Ово се чита као: у колони 0 узми ред 0, у колони 1 узми ред 1, у колони 2 узми ред 2, у колони 3 узми ред 1, у колони 4 узми ред 0.

Визуелно, за видљиви прозор димензија 3×5 :

	Кол. 1	Кол. 2	Кол. 3	Кол. 4	Кол.5
Ред 0:	[X]	[]	[]	[]	[X]
Ред 1:	[]	[X]	[]	[X]	[]
Ред 2:	[]	[]	[X]	[]	[]

Поступак евалуације једне путање

За дату путању и дати видљиви прозор, евалуација се одвија у три корака.

Корак 1: Читање токена са позиција. Алгоритам пролази кроз путању и из сваке колоне чита токен на реду који путања дефинише. Резултат је евалуациона секвенца – низ од n токена.

Видљиви прозор:

	Кол. 1	Кол. 2	Кол. 3	Кол. 4	Кол. 5
Ред 0:	A	B	D	A	C
Ред 1:	B	A	A	B	A
Ред 2:	A	D	B	A	B

Путања: [0, 1, 2, 1, 0]

Читање:

Колона 0, ред 0 $\rightarrow$ A

Колона 1, ред 1 $\rightarrow$ A

Колона 2, ред 2 $\rightarrow$ B

Колона 3, ред 1 $\rightarrow$ B

Колона 4, ред 0 $\rightarrow$ C

Евалуациона секвенца: [A, A, B, B, C]

Корак 2: Бројање узастопних поклапања од почетка (префикс). Алгоритам утврђује који токен се поклапа на почетку секвенце и броји колико узастопних позиција садржи тај токен (или универзални заменски токен). Бројање се прекида чим се наиђе на токен који се не поклапа.

Секвенца: [A, A, B, B, C]

Позиција 0: A – поклапање (токен који се тражи је A)

Позиција 1: A – поклапање

Позиција 2: B – није A нити заменски токен → прекид

Резултат: токен A, дужина поклапања = 2

Ако се на почетку секвенце налази универзални заменски токен σ_w , алгоритам тражи први токен који није σ_w , и користи га као референтни токен за поклапање. Ако су сви токени у секвенци σ_w , цела секвенца се третира као поклапање заменског токена.

Секвенца [σ_w , σ_w , B, A, C]

Позиција 0: σ_w – поклапање (заменски)

Позиција 1: σ_w – поклапање (заменски)

Позиција 2: B – први прави токен, постаје референтни (поклапање B = B)

Позиција 3: A – није B нити σ_w → прекид

Резултат: токен B, дужина поклапања = 3

Корак 3: Претрага евалуационе табеле. На основу токена и дужине поклапања, алгоритам чита излазну вредност из евалуационе табеле. Ако је дужина поклапања испод минималног прага (обично 3), излазна вредност је 0.

Поступак евалуације свих путања

Алгоритам понавља горњи поступак за сваку од L путања и сабира све позитивне излазне вредности.

Псеудо код

```
funkcija Evaluiraj(prozor, putanje, f):
    ukupnaVrednost  $\leftarrow$  0
    za svaku putanju p u P:
        sekvenca  $\leftarrow$  PročitajTokenSaPutanje(prozor, putanja)
        token  $\leftarrow$  PrviNezamenski(sekvenca)
        dužina  $\leftarrow$  BrojUzastopnih(sekvenca, token)
        ukupnaVrednost  $\leftarrow$  ukupnaVrednost + f(token, dužina)
    vrati ukupnaVrednost

funkcija PročitajTokenSaPutanje(prozor, putanja):
    sekvenca  $\leftarrow$  prazan niz dužine N
    za j od 0 do N-1:
        sekvenca[j]  $\leftarrow$  W[p[j]][j]
    vrati sekvenca

funkcija PrviNezamenski(sekvenca);
    za i od 0 do N-1:
        ako sekvenca[i]  $\neq$   $\sigma_w$  i sekvenca[i]  $\neq$   $\sigma_s$ :
            vrati sekvenca[i]
    vrati  $\sigma_w$ 

funkcija BrojUzastopnih(sekvenca, token):
    brojač  $\leftarrow$  0
    za i od 0 do N-1:
        ako sekvenca[i] = token ili sekvenca[i] =  $\sigma_w$ :
            brojač  $\leftarrow$  brojač + 1
        inače
            prekini
    vrati brojač
```

C# код

```
List<LineWin> Evaluate(char[][] matrix)
{
    var wins = new List<LineWin>();

    for (int lineIdx = 0; lineIdx < Lines.Length; lineIdx++)
    {
        char[] sequence = ReadTokensFromLine(matrix, Lines[lineIdx]);
        char token = FirstNonReplaceable(sequence);
        int count = CountConsecutive(sequence, token);

        if (Paytable[token].TryGetValue(count, out var value))
            wins.Add(new Result(lineIdx, token, count, value));
    }
    return wins;
}

char[] ReadTokensFromLine(char[][] matrix, int[] line)
{
    char[] sequence = new char[Reels];
    for (int reel = 0; reel < Reels; reel++)
        sequence[reel] = matrix[line[reel]][reel];

    return sequence;
}

char FirstNonReplaceable(char[] sequence)
{
    for (int reel = 0; reel < Reels; reel++)
        if (sequence[reel] != Wild && sequence[reel] != Scatter)
            return sequence[reel];

    return Wild;
}

int CountConsecutive(char[] sequence, char token)
{
    int count = 0;
    for (int reel = 0; reel < Reels; reel++)
    {
        if (sequence[reel] == token || sequence[reel] == Wild)
            count++;
        else
            break;
    }
    return count;
}
```

Анализа сложености

За сваку итерацију система, алгоритам пролази кроз свих L евалуационих путања. За сваку путању чита n позиција из прозора у најгорем случају (када се поклапање протеже до последње колоне). У просечном случају, петља за бројање се прекида раније јер се токени не поклапају, али асимптотски најгори случај остаје:

$$T_{seq} = O(L \cdot n)$$

За систем са на пример $L = 40$ путања и $n = 5$ колона, то је до 200 операција читања и поређења по итерацији. Меморијска сложеност је минимална – потребан је само дводимензионални низ путања (фиксна структура која се не мења) и сам видљиви прозор.

Предност: једноставна имплементација, минимална меморија, нема фазе припреме, лако разумљив код.

Недостаци: свака путања се евалуира потпуно независно. Ако две путање деле исте позиције на првим колона (нпр. путање (0, 0, 0, 0, 0) и (0, 0, 0, 1, 2) деле прве три колоне), алгоритам то не препознаје – исти токени се читају и пореде два пута. За $L = 40$ путања на једном од $m = 4$ реда, у просеку 10 путања дели исту почетну позицију, а алгоритам за сваку од њих независно чита исти токен са те позиције.

5.3. Евалуација помоћу хеш табеле

Опис приступа

Идеја овог приступа је да се целокупна логика евалуационе функције изврши унапред, пре него што систем почне са радом. За сваку могућу комбинацију од n токена израчуна се излазна вредност. Комбинације које дају позитивну вредност сместе се у хеш табелу. Током евалуације, за сваку путању се формира кључ од n токена и претражи табела – уместо да се за сваку путању поново броје узастопна поклапања.

Приступ се састоји од две фазе: једнократна фаза припреме (изградња табеле) и фаза евалуације (претрага табеле).

Фаза припреме – изградња хеш табеле

Алгоритам генерише све могуће комбинације од n токена из азбуке токена. За сваку комбинацију примени евалуациону функцију (исту логику бројања префикса као у секвенцијалном приступу) и ако је излазна вредност позитивна, упише ту комбинацију у хеш табелу.

Прављење кључа

Да би се n токена ефикасно користило као кључ хеш табеле, токени се пакују у један целобројни тип. Сваки токен се репрезентује као 8-бетна вредност (један бајт – довољно за азбуку до 256 токена). За систем од на пример пет колона, пет токена се смешта у један 64-битни цео број тако што се сваки следећи токен помера 8 бита улево:

Токен:	A	B	D	A	C
ASCII:	65	66	68	65	67
Позиција:	0-7	8-15	16-23	24-31	32-39

Кључ = $65 \mid (66 \ll 8) \mid (68 \ll 16) \mid (65 \ll 24) \mid (67 \ll 32)$

Овим се пет токена претварају у јединствени број који служи као кључ за претрагу. Два различита низа токена никад не дају исти кључ јер сваки токен заузима своју фиксну позицију унутар 64-битног броја.

Фаза евалуације – претрага табеле

За сваку евалуациону путању, алгоритам:

1. Чита токене са позиција које путања дефинише (исто као у секвенцијалном приступу)
2. Пакује тих n токена у 64-битни кључ.
3. Претражује хеш табелу за тај кључ.
4. Ако кључ постоји, излазна вредност се чита из табеле. Ако не постоји, излазна вредност је 0.

Суштинска разлика у односу на секвенцијални приступ: нема петље за бројање поклапања, нема тражења првог незаменског токена, нема гранања – само једно рачунање кључа и претрага табеле.

Псеудо код

```
funkcija IzgradiHesTabelu(azbuka, n, f):
    tabela ← nova HešMapa
    za svaku kombinaciju iz azbuka:
        token ← PrviNezamenski(kombinacija)
        dužina ← BrojUzastopnih(kombinacija, token)
        vrednost ← f(token, dužina)
        ako vrednost > 0:
            ključ ← PravljenjeKljuča(kombinacija)
            tabela[ključ] ← (token, dužina, vrednsot)
    vrati tabela

funkcija PravljenjeKljuča(sekvenca)
    ključ ← 0
    za i od 0 do n-1:
        ključ ← ključ | (sekvenca[i] << (i * 8))
    vrati ključ

funkcija Evaluiraj(W, P, tabela):
    izlaz ← 0
    za svaku putanju p u P:
        simbol_1 ← W[p[0]][0]
        simbol_2 ← W[p[1]][1]
        ...
        simbol_n ← W[p[n-1]][n-1]
        ključ ← PravljenjeKljuča(simbol_1, ..., simbol_n)
        ako tabela sadrži ključ:
            izlaz ← izlaz + tablea[ključ].vrednost
    vrati izlaz
```

Имплементација у С#

```
private static long PackKey(char s0, char s1, char s2, char s3, char s4) ⇒
    s0 | ((long)s1 << 8) | ((long)s2 << 16) | ((long)s3 << 24) | ((long)s4 << 32);
```

```

private static Dictionary<long, (char Symbol, int Count, int Payout)> BuildWinDictionary()
{
    var dict = new Dictionary<long, (char Symbol, int Count, int Payout)>();
    char[] allSymbols = ['S', '7', 'B', 'M', 'P', 'L', 'O', 'C'];

    foreach (var s0 in allSymbols)
    foreach (var s1 in allSymbols)
    foreach (var s2 in allSymbols)
    foreach (var s3 in allSymbols)
    foreach (var s4 in allSymbols)
    {
        var result = EvaluateSingleLine(s0, s1, s2, s3, s4);
        if (result.Payout > 0)
            dict[PackKey(s0, s1, s2, s3, s4)] = result;
    }

    return dict;
}

```

```

private static List<LineWin> EvaluateLinesDictionary(char[][] matrix)
{
    var wins = new List<LineWin>();

    for (int lineIdx = 0; lineIdx < PlentyOfFruitConfig.Lines.Length; lineIdx++)
    {
        var line = PlentyOfFruitConfig.Lines[lineIdx];
        var key = PackKey(
            matrix[line[0]][0],
            matrix[line[1]][1],
            matrix[line[2]][2],
            matrix[line[3]][3],
            matrix[line[4]][4]
        );

        if (WinDictionary.TryGetValue(key, out var win))
            wins.Add(new Result(lineIdx, win.Symbol, win.Count, win.Value));
    }

    return wins;
}

```

Анализа сложености

Фаза припреме: Генерисање свих k^n комбинација и евалуација сваке је $O(k^n \cdot n)$. За $k = 8$ и $n = 5$: $32\,768 \times 5 = 163\,840$ операција. Ово се извршава једнократно при покретању система.

Фаза евалуације: За сваку путању паковање кључа је $O(n)$ (5 bit-shift операција и 5 OR операција), а за претрагу хеш табеле је $O(1)$ амортизовано. Укупно по итерацији:

$$T_{\text{hash}} = O(L \cdot n)$$

Асимптотски, ово је иста сложеност као секвенцијална евалуација. Разлика је у томе шта се раду унутар тих $L \cdot n$ операција – секвенцијални приступ чита токене, пореди их и броји поклапања са гранањем, док хеш приступ чита токене, пакује их и ради једну претрагу табеле.

Меморијска сложеност: Хеш табела садржи највише k^n уноса. За $k=8$ и $n=5$ то је 32 768 уноса – у пракси мање јер се чувају само комбинације са позитивном вредношћу. За веће системе меморија расте експоненцијално: $k=15$ и $n=6$ даје $15^6 \approx 11.4$ милиона уноса.

Предности: Евалуација се своди на претрагу табеле; логика евалуационе функције се извршава само једном у фази припреме; лако додавање нових правила.

Недостаци: Меморијски отисак расте експоненцијално са n и k ; путање се и даље евалуирају независно без дељења заједничких префикса; за мале n трошкови прављења кључа и хеширања могу поништити предности избегавања петље за бројање.

5.4. Евалуација помоћу индексног стабла

Кључно запажање које мотивише овај приступ је следеће: евалуационе путање нису независне. Многе путање деле исте позиције на почетним колонама.

На примеру, у систему са $m = 4$ реда, све путање које почињу од реда 0 у првој колони читају исти токен на тој позицији. Ако је тај токен позиционо-инваријантан, ниједна од тих путања не може дати валидан образац – али секвенцијални приступ и хеш приступ то проверавају за сваку путању посебно.

Индексно стабло (trie) организује евалуационе путање у структурно стабло где се заједнички префикси деле. Чворови стабла одговарају позицијама у видљивом прозору, а гране одговарају преласку на следећу колону. Путање које се поклапају на првим i колонама деле исти пут од корена до i -тог нивоа стабла.

Структура стабла

Стабло има n нивоа, где сваки ниво одговара једној колони видљивог прозора. Сваки чвор на нивоу i садржи:

- Индекс реда – који ред видљивог прозора овај чвор представља на својој колони.
- Низ деце – показивачи на чворове следећег нивоа, индексирани по реду (величине m). Ако ниједна путања не пролази кроз дати ред на следећој колони, одговарајући показивач је празан.
- Листа завршених путања – индекси путања које се завршавају у овом чвору (присутно само на последњем нивоу стабла).

Корен стабла није један чвор него низ од m коренских чворова – по један за сваки могући ред у првој колони.

Псеудо код изградње стабла:

```
funkcija IzgradiStablo(P, m, n):
    koreni ← niz od m praznih pokazivača
    za svaku putanju p sa indeksom l u P:
        početniRed ← p[0]
        ako koreni[početniRed] ne postoji:
            koreni[početniRed] ← noviČvor(red = početniRed)

        trenut ← koreni[početniRed]
        za kolonu od 1 do n-1:
            slRed ← p[kolonu]
            ako trenut.deca[slRed] ne postoji:
                trenut.deca[slRed] ← noviČvor(red = slRed)
            trenut ← trenut.deca[slRed]

        trenut.završenePutanje.dodaj(l)

    vrati koreni
```

Имплементација чвора (C#)

```
public class LineTrieNode
{
    public int Row { get; init; }
    public List<int> CompletedLineIndices { get; } = [];
    public LineTrieNode?[] Children { get; } = new LineTrieNode[Rows];
}
```

Имплементација изградње стабла (C#)

```
private static LineTrieNode[] BuildLineTrie()
{
    LineTrieNode[] roots = new LineTrieNode[Rows];

    for (int lineIdx = 0; lineIdx < Lines.Length; lineIdx++)
    {
        int[] line = Lines[lineIdx];
        int startRow = line[0];

        if (roots[startRow] == null)
        {
            roots[startRow] = new LineTrieNode();
            roots[startRow].Row = startRow;
        }

        LineTrieNode currentNode = roots[startRow];

        for (int col = 1; col < Reels; col++)
        {
            int nextRow = line[col];

            if (currentNode.Children[nextRow] == null)
            {
                LineTrieNode newNode = new LineTrieNode();
                newNode.Row = nextRow;

                currentNode.Children[nextRow] = newNode;
            }

            currentNode = currentNode.Children[nextRow];
        }

        int lineId = lineIdx + 1;
        currentNode.CompletedLineIndices.Add(lineId);
    }

    return roots;
}
```

Визуелни пример

За систем са $m = 4$ реда, $n = 5$ колона и 4 путање:

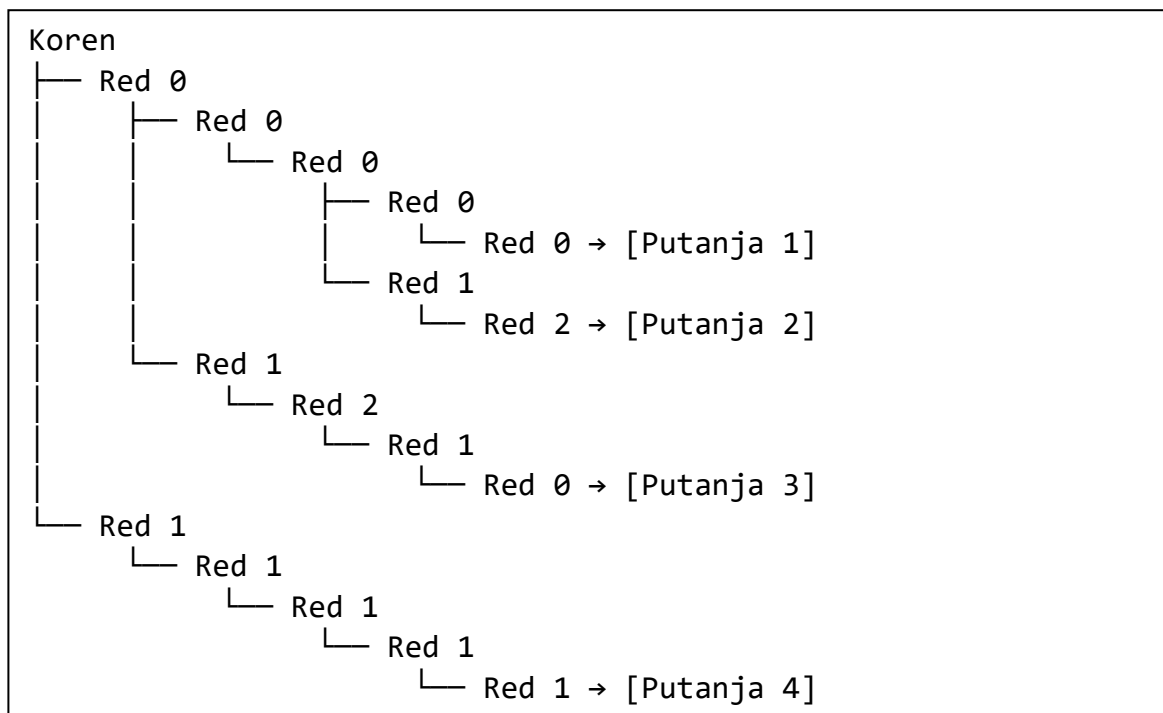
Путања 1: $(0, 0, 0, 0, 0)$

Путања 2: (0, 0, 0, 1, 2)

Путања 2: (0, 1, 2, 1, 0)

Путања 4: (1, 1, 1, 1, 1)

Стабло изгледа овако:



Путање 1 и 2 деле прва три нивоа стабла (обе почињу са редом 0 на прве три колоне). Када алгоритам обрађује ту грану, токен на позицији (колона 0, ред 0) чита се само једном – не два пута као код секвенцијалног приступа.

Пример евалуације обиласком стабла

Користимо исто стабло из претходног примера (4 путање) и конкретан видљиви прозор:

Видљиви прозор:

	Кол. 0	Кол. 1	Кол. 2	Кол. 3	Кол. 4
Ред 0:	A	A	A	B	C
Ред 1:	B	σ_s	A	A	A
Ред 2:	C	A	B	A	B
Ред 3:	A	B	C	C	A

Алгоритам креће од коренских чворова и обрађује сваки:

Корен Ред 0 – токен на позицији (Кол. 0, Ред 0) = А

Токен није позиционо-инваријантни, настављамо. Референтни токен за поклапање: А.

Алгоритам улази у грану Ред 0 → Ред 0 (Колона 1, Ред 0):

Ниво 0: Кол. 0, Ред 0 → А поклапање (А=А)

Ниво 1: Кол. 1, Ред 0 → А поклапање (А=А)

Из овог чвора постоје две гране – Ред 0 на колони 2:

Ниво 2: Кол. 2, Ред 0 → А поклапање (А=А)

Сада грана ка Ред 0 на колони 3:

Ниво 3: Кол. 3, Ред 0 → В није А нити σw → прекид поклапања

Дужина поклапања је 3 (колони 0, 1, 3). Функција излазне вредности $f(A, 3) = 5$. Ова грана представља путању 1 – алгоритам додаје излазну вредност 5 за путању 1.

Враћамо се на ниво 2 и улазимо у другу грану (Ред 1 на колони 3):

Ниво 3: Кол. 3, Ред 1 → А поклапање (А=А)

Ниво 4: Кол. 4, Ред 2 → Б није А нити σw → прекид поклапања

Дужина поклапања је 4 (колони 0, 1, 2, 3). Функција излазне вредности $f(A, 4) = 20$. Ова грана представља путању 2 – алгоритам додаје излазну вредност 20 за путању 2.

Сада се алгоритам враћа на корен Ред 0 и улази у другу грану – Ред 1 на колони 1:

Ниво 1: Кол. 1, Ред → σs позиционо-инваријантни токен → одсецање

Цела грана се прескочи. Путања 3, која пролази кроз ову грану, се елиминише једном провером. Алгоритам не улази дубље у стабло и не чита токене на колонама 2, 3 и 4 за ову путању.

Корен Ред 1 – токен на позицији (Кол. 0, Ред 1) = В

Референтни токен: В. Алгоритам улази у грану Ред 1 на колони 1:

Ниво 1: Кол. 1, Ред 1 → σs позиционо инваријантни токен → одсецање

Поново одсецање. Путања 4 се елиминише једном провером.

Преглед целокупне евалуације:

Путања 1: евалуирана, префикс А дужине 3 → излазна вредност = 5

Путања 2: евалуирана, префикс А дужине 4 → излазна вредност = 20

Путања 3: елиминисана одсецањем на колони 1

Путања 4: елиминисана одсецањем на колони 1

Укупна излазна вредност: 25

Поређење са секвенцијални приступом на истом примеру:

	Секвенцијална	Индексно стабло
Читање токена из прозора	20 (4 путања $\times$ 5 колона)	9
Поређење токена	20	7
Евалуиране путање до краја	4	2

Секвенцијални приступ чита свих 20 токена јер обрађује сваку путању независно. Индексно стабло чита само 9 токена, колону 0 чита два пута (корен Ред 0 и Ред 1 уместо четири пута), дели читање колоне 1 између путања 1 и 2, и одсеца путање 3 и 4 на колони 1 без читања колона 2-4.

У реалном систему са $L = 40$ путања, ефекат је израженији. На првој колони, уместо 40 читања, стабло чита само $m = 4$ токена (по један за сваки коренски чвор). Свако одсецање на раним нивоима елиминише у просеку $L/m = 10$ путања одједном.

Евалуација обиласком стабла

Евалуација се врши рекурзивним обиласком стабла. На сваком нивоу, алгоритам чита токен из видљивог прозора на позицији коју чвор дефинише и проверава да ли се поклапања наставља. Кључна предност је могућност **одсецања грана** (pruning): Ако се на неком нивоу појави токен прекида поклапање, цела грана се прескочи као и све путање које пролазе кроз ту грану се елиминишу одједном.

Псеудо код евалуације:

```
funkcija EvaluirajStablom(W, koreni, f):  
    ukupnaVrednost ← 0  
    za svaki korenRed od 0 do m-1:  
        ako koreni[korenRed] ne postoji: preskoči  
        prviToken ← W[korenRed][0]  
        ako prviToken = 0: preskoči  
        Obiđi(W, koreni[korenRed], kolona=0, prviToken, f, ukupnaVrednost)  
    vrati ukupnaVrednost  
  
funkcija Obiđi(W, čvor, kolona, trenutniToken, f, ukupnaVrednost):  
    dužina ← kolona + 1  
    ako čvor.završenePutanje nije prazan:  
        vrednost ← f(trenutniToken, dužina)  
        za svaku putanju u čvor.završenePutanje:  
            ukupnaVrednost ← ukupnaVrednost + vrednost  
  
    ako kolona >= n-1: vreti  
  
    za sledećiRed od 0 do m-1:  
        ako čvor.deca[sledećiRed] ne postoji: preskoči  
        sledećiToken ← W[sledećiRed][kolona + 1]  
        ako sledećiToken = 0: preskoči  
        ako sledećiToken = trenutniToken ili sledeći token = 0:  
            Obiđi(W, čvor.deca[sledećiRed], kolona+1, trenutniToken, f,  
ukupnaVrednost)  
        inače:  
            vrednost ← f(trenutniToke, dužina)  
            ako vrednost > 0  
                PrikupiSvePutanjeIzGrane(čvor.deca[sledećiRed],  
vrednost, ukupnaVrednost)
```

Имплементација евалуације (C#)

```
private static void TraverseTrieForWins(
    char[][] matrix, LineTrieNode node, int reel,
    char matchSymbol, bool allWildSoFar, List<LineWin> wins
)
{
    int consecutiveCount = reel + 1;
    if (node.CompletedLineIndices.Count > 0)
    {
        char paySymbol = allWildSoFar ? Wild : matchSymbol;
        if (Paytable.TryGetValue(paySymbol, out var payouts)
            && payouts.TryGetValue(consecutiveCount, out var payout))
        {
            foreach (var lineIndex in node.CompletedLineIndices)
                wins.Add(new LineWin());
        }
    }
    if (reel ≥ Reels - 1) return;
    for (int nextRow = 0; nextRow < Rows; nextRow++)
    {
        var child = node.Children[nextRow];
        if (child == null) continue;
        char nextSymbol = matrix[nextRow][reel + 1];
        if (nextSymbol == Scatter) continue;
        bool nextIsWild = nextSymbol == Wild;
        bool matches = nextIsWild || nextSymbol == matchSymbol || allWildSoFar;
        if (!matches)
        {
            char paySymbol = allWildSoFar ? Wild : matchSymbol;
            if (Paytable.TryGetValue(paySymbol, out var payouts)
                && payouts.TryGetValue(consecutiveCount, out var payout))
            {
                CollectAllLineIndices(child, wins, consecutiveCount, paySymbol, payout);
            }
            continue;
        }
        char newMatchSymbol = (allWildSoFar && !nextIsWild) ? nextSymbol : matchSymbol;
        bool newAllWild = allWildSoFar && nextIsWild;
        TraverseTrieForWins(matrix, child, reel + 1, newMatchSymbol, newAllWild, wins);
    }
}
```

Анализа сложености

Фаза припреме: Изградња стабла пролази кроз сваку путању једном и за сваку путању уписује n чворова. Сложеност:

$$T_{\text{init}} = O(L \cdot n)$$

Фаза евалуације – најгори случај: У најгорем случају (све путање су потпуно различите и сваки токен се поклапа), стабло дегенерира у секвенцијалну евалуацију:

$$T_{\text{worst}} = O(L \cdot n)$$

Фаза евалуације – просечан случај: У пракси, одсецање значајно редукује број операција. Када се грана одсече на нивоу i , све путање које пролазе кроз ту грану (а може их бити много) се елиминишу без проласка кроз преосталих $n - i$ нивоа. Ефикасност одсецања зависи од:

- Степена дељења префикса – што више путања дели почетне позиције, стабло је компактније и једно одсецање елиминише више путања.
- Дистрибуције токена – ако се на раним колонама појаве токени који прекидају поклапање, одсецање наступа рано и елиминишу дубоке гране.
- Густине позиционо-инваријантних токена – свако појављивање овог токена на позиција корена елиминише све путање из те гране једном провером.

Меморијска сложеност: Стабло држи највише $L \cdot n$ чворова (када ниједна путања не дели ни један префикс), а у пракси мање. Сваки чвор садржи низ показивача величине m и листу индекса завршених путања.

5.5. Компаративна анализа

Теоријско поређење

Карактеристика	Секвенцијална	Хеш табела	Индексно стабло
Сложеност припреме	Без припреме	$O(k^n \cdot n)$	$O(L \cdot n)$
Сложеност евалуације (најгори случај)	$O(L \cdot n)$	$O(L \cdot n)$	$O(L \cdot n)$
Сложеност евалуације (просечан случај)	$O(L \cdot n)$	$O(L \cdot n)$	$O(L' \cdot n)$, где је $L' < L$
Меморија	Без додатне меморије	$O(k^n)$	$O(L \cdot n \cdot m)$

Ознака L' у просечном случају индексног стабла означава ефективан број путања које алгоритам заправо евалуира, након одсецања. У пракси L' може бити значајно мање од L .

Експериментално поређење:

Све три методе тестиране су на идентичном систему са следећим параметрима:

- Скуп токена: $k = 8$ (6 стандардних токена, 1 заменски, 1 позиционо-инваријантан)
- Димензије прозора: $m = 4$ реда, $n = 5$ колона
- Број евалуационих путања: $L = 40$
- Дужине токен-секвенци: између 337 и 393 по колони
- Процесор: AMD Ryzen 5 7520U
- Платформа: .NET 10, Release конфигурације

Резултати

Број итерација	Секвенцијална	Хеш табела	Индексно стабло
1 000 000	1 972 ms	1 678 ms	1 005 ms
10 000 000	15 440 ms	15 452 ms	9 996 ms
100 000 000	145 170 ms	148 801 ms	97 259 ms

Метода	Време припреме
Секвенцијална	/
Хеш табела	4 ms
Индексно стабло	< 1 ms

Релативне перформансе

Ако секвенцијалну евалуацију узмемо као базну линију (1.00x), релативне брзине су:

Број итерација	Секвенцијална	Хеш табела	Индексно стабло
1 000 000	1.00x	1.18x	1.96x
10 000 000	1.00x	1.00x	1.54x
100 000 000	1.00x	0.98x	1.49x

Анализа резултата

Секвенцијална евалуација vs. хеш табела. Упркос теоријској предности (претрага табеле уместо итеративног бројања), хеш табела не односи значајно убрзање. На 100 милиона итерација хеш приступ је чак маргинално спорији. Разлог је у томе што је евалуација по префиксу само по себи врло ефикасна – унутрашња петља броји узастопна поклапања и прекида при првом неслагању, што је у просеку мање од n операција. Хеш приступ замењује ову петљу претрагом табеле, али уводи додатне трошкове: прављење кључа, израчунавање хеш вредности, и потенцијалне колизије у табели. За мале вредности n ови трошкови поништавају предности.

Индексно стабло: Конзистентна предност индексног стабла произлази из две особине које дуга два приступа немају:

1. **Дељење префикса:** У конкретном систему са 40 путања и 4 могућа реда на првој колони, постоје само 4 коренска чвора. То значи да се токен на првој колони чита највише 4 пута уместо 40 пута. На сваком следећем нивоу дељење се наставља – стабло има много мање чворова него $L \cdot n$.
2. **Одсецање грана:** Када се на некој позицији појави токен који прекида поклапање, цела грана стабла се прескочи. Једна провера елиминише све путање у тој грани.

Скалабилност: Предност индексног стабла расте са бројем евалуационих путања L је већи L значи више прилика за одсецање.

5.6. Закључак поглавља

Од три анализирана приступа, индексно стабло даје најбоље перформансе у пракси. Његова предност не потиче од боље асимптотске сложености у најгорем случају (сва три приступа су $O(L \cdot n)$), већ од способности да експлоатише структуру евалуационих путања – дељење заједничких префикса и рано одсецање грана које не могу дати валидан образац.

Хеш табела, упркос интуитивној привлачности приступа „прерачунај све унапред“, не доноси значајно убрзање јер је сама евалуациона функција (бројање узастопних поклапања) довољно ефикасна да трошкови хеширања не буду оправдани.

За системе са великим бројем евалуационих путања и за сценарије где је потребно извршити велики број итерација, индексно стабло је препоручен приступ.

6. Литература

Cormen, Thomas H. / Leiserson, Charles E. / Rivest, Ronald L. / Stein, Clifford. Introduction to Algorithms. MIT Press. 2009.

Feller, William. An Introduction to Probability Theory and Its Applications, Vol. 1. John Wiley & Sons. 1968.

Knuth, Donald E. Seminumerical Algorithms. The Art of Computer Programming, Vol. 2. Addison-Wesley. 1997.

Compeau, Phillip E.C. / Pevzner, Pavel A. / Tesler, Glenn. How to apply de Bruijn graphs to genome assembly. Nature Biotechnology, Vol. 29, No. 11, стр. 987–991. 2011.

Pevzner, Pavel A. / Tang, Haixu / Waterman, Michael S. An Eulerian path approach to DNA fragment assembly. Proceedings of the National Academy of Sciences, Vol. 98, No. 17, стр. 9748–9753. 2001.

Nocedal, Jorge / Wright, Stephen J. Numerical Optimization. Springer. 2006.

Kraft, Dieter. A Software Package for Sequential Quadratic Programming. Deutsche Forschungs- und Versuchsanstalt für Luft- und Raumfahrt. 1988.

Fredkin, Edward. Trie Memory. Communications of the ACM, Vol. 3, No. 9, стр. 490–499. 1960.

Hierholzer, Carl / Wiener, Chr. Ueber die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren. Mathematische Annalen, Vol. 6, стр. 30–32. 1873.

Harris, C.R. / Millman, K.J. / van der Walt, S.J. и др. Array programming with NumPy. Nature, Vol. 585, стр. 357–362. 2020.

Virtanen, P. / Gommers, R. / Oliphant, T.E. и др. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. Nature Methods, Vol. 17, стр. 261–272. 2020.

Balinski, Michel L. / Young, H. Peyton. Fair Representation: Meeting the Ideal of One Man, One Vote. Brookings Institution Press. 2001.